

Chapter 9

Contents

Chapters

- 9. Timers & its Programming
 - 9.1 Overview of Timers
 - 9.2 Timer structures & its programming
 - 9.2.1 Timer 0
 - 9.2.2 Timer 1
 - 9.2.3 Timer 2
 - 9.2.4 Timer 3
 - 9.2.5 Timer 4
 - 9.3 Using Timers as counters
 - 9.4 Using Timers to determine number of instruction cycles

9.1 Overview of Timers

Timers are an important part of microcontrollers and are required for various applications. Timers typically are used to count the number of clock pulses that are received by the timer module, and they are designed to work as timer or counters.

A timer that is sourced from a periodic clock source coming from a crystal oscillator that is internal or external to a microcontroller is regularly called as a timer. Timers could be used for various applications, and a typical application example is the square wave generator. Timers are also used with CCP modules to capture a signal, compare and generate PWM signals and control the sampling rate for A/D conversion. Some applications include determining the frequency of an input signal, finding the width of the pulse along with generating a waveform.

Timers may also be sourced from devices (ex: sensors) that generate periodic or non-periodic pulses based on application. Timers used for such applications are typically called as counters. A typical application of a counter is measurement of speed by counting the number of pulses generated by a proximity sensor.

Program 9.1: An ALP to create a delay without using a Timer

The following example demonstrates creation of delay without using a timer.

No	Label	Op-Code	Operand	Comments
		#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
		ORG	0x0000	; Set Origin 0x0000 of Flash memory
1		GOTO	START	; Go to Start the Main program.
		ORG	0x0100	; Set Origin 0x0100 for Main Program
2	START	MOVLW	0x03	; WREG = 0x03
3		MOVWF	0x11,0	; [0x11] = WREG = 0x03, Variable y
4	L2:	MOVLW	0x04	; WREG = 0x04
5		MOVWF	0x10,0	; [0x10] = WREG = 0x04, Variable x
6	L1:	DECF	0x10, F	; Decrement File Reg 0x10, [0x10] = [0x10] - 1
7		BNZ	L1	; If not zero Branch to L1.
8		DECF	0x11, F	; Decrement File Reg 0x11, [0x11] = [0x11] - 1
9		BNZ	L2	; If not zero Branch to L2.
10		NOP		; No Operation
		END		

The number of cycles required for the respective instruction in the program is shown in Fig P9.1a and the respective instruction wise execution and timing of the program is shown in Fig P9.1b.

The ideal execution cycles for looping instructions are a multiplication of content in file register 0x10 and 0x11, i.e.

$$T_{\text{ideal}} = x * y = 0x04 * 0x03 = 12d = 0x0C.$$



Instruction No	Label	Op-Code	Operand	Cycles
		ORG	0x0000	0
1		GOTO	START	2
		ORG	0x0100	0
2	START	MOVLW	0x03	1
3		MOVWF	0x11,0	1
4	L2:	MOVLW	0x04	1
5		MOVWF	0x10,0	1
6	L1:	DECF	0x10,F	1
7		BNZ	L1	2
8		DECF	0x11,F	1
9		BNZ	L2	2
10		NOP		1
		END		0

Fig P9.1a: Program demonstrating cycles used, loop areas.

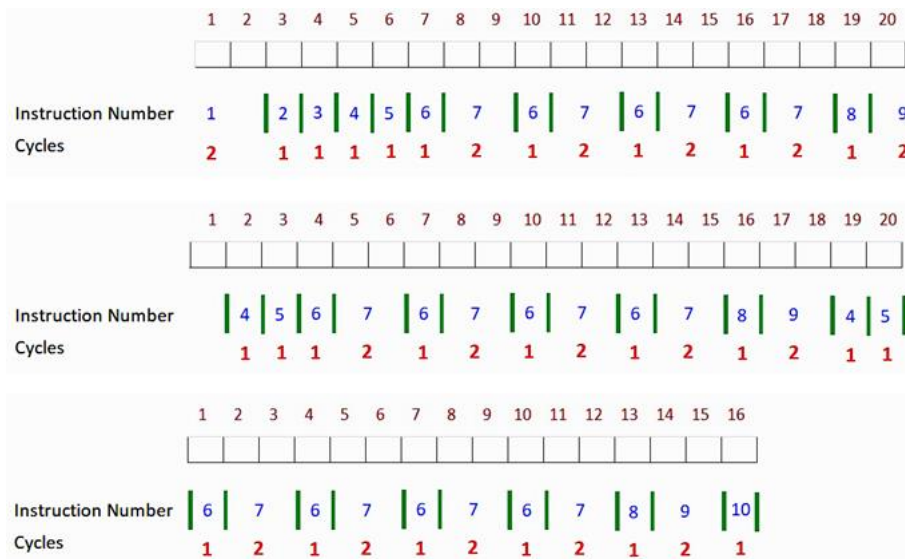


Fig P9.1b: Instruction wise execution & timing of the program.

Actually, the number of cycles that are taken varies from the ideal execution cycles. The practical execution cycles taken for the complete program is given by

$$T_{\text{practical}} = 4 + \{5 + 3 * 0x04\} * 0x03 + 1 = 56d = 0x38$$

The looping instructions i.e. instruction 4 to 9 contributes 51 cycles (i.e. the actual delay cycles) and the other instructions 1, 2, 3 and 10 contribute 5 cycles.

The split is given as $2+1+1 + \{1+1 + [1+2] * 0x04 + 1+2\} * 0x03 + 1$.

i.e. $2 \underline{(1)} + 1 \underline{(2)} + 1 \underline{(3)} + \{ \{ 1 \underline{(4)} + 1 \underline{(5)} + [1 \underline{(6)} + 2 \underline{(7)}] * 0x04 + 1 \underline{(8)} + 2 \underline{(9)} \} * 0x03 + 1 \underline{(10)}$

The underlined numbers enclosed by parenthesis are the instruction numbers. The values of x and y may be fine-tuned to get the required number of delay cycles. For example, to get close to 12 cycles x is to be 1 and y is to be 1 which gives practical cycles of 13 cycles. Similarly, for a 21 cycles x is to be 1 and y is to be 2 and so on.

The Fig P9.1b shows total of 56 cycles with first two rows being numbered from 1 to 20 and the third numbered 1 to 16 making a total of 56 cycles required for execution of the program.

Program 9.2: Generate a square wave that creates a delay without using a Timer using modular approach.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0050	; Set Origin 0x0000 of Flash memory
DLY	MOVLW	0x03	; WREG = 0x03
	MOVWF	0x11,0	; [0x11] = WREG = 0x03
L2:	MOVLW	0x04	; WREG = 0x04
	MOVWF	0x10,0	; [0x10] = WREG = 0x04
L1:	DECF	0x10,F	; Decrement File Register 0x10, [0x10] = [0x10] - 1
	BNZ	L1	; If not zero Branch to L1.
	DECF	0x11,F	; Decrement File Register 0x11, [0x11] = [0x11] - 1
	BNZ	L2	; If not zero Branch to L2.
	RETURN		; Return from Delay module
	ORG	0x0100	; Set Origin 0x0100 for Main Program
START	BCF	TRISB,0	; Set PortB Pin 0 as Output, TRISB.0=0
M1:	BTG	PORTB,0	; Toggle PortB.0
	CALL	DLY	; Call Delay Module to set Pulse Width
	BRA	M1	; Branch to M1
	END		

This program makes use of subroutine that is called in the main program to create the required delay. The ideal execution cycles for the delay is multiplication of content in file register 0x10 & 0x11, i.e. The numbered instructions & cycles taken is shown in Fig P9.2.

$$T_{ideal} = x * y = 0x04 * 0x03 = 12d = 0x0C.$$

Actually, the number of cycles that are taken varies from the ideal execution cycles. The practical execution cycles for the complete program is given by

$$T_{practical} = 8 + DLY = 63d = 0x3F$$

The split for one full program iteration is given as $2 + 1 + 1 + 2 + DLY + 2$.

i.e $2(1) + 1(11) + 1(12) + 2(13) + DLY + 2(14)$.

The delay created by DLY module is given as

$$1 + 1 + \{1 + 1 + [1 + 2] * 0x04 + 1 + 2\} * 0x03 + 2 = 55 \text{ cycles}$$

i.e $1(2) + 1(3) + \{ \{ 1(4) + 1(5) + [1(6) + 2(7)] * 0x04 + 1(8) + 2(9) \} * 0x03 + 2(10)$

This brings the number of execution cycles for a single iteration of full program to 63 cycles.

The delay module part i.e. instruction 2 to 10 contributes 55 cycles and the other instructions 1, 11, 12, 13 and 14 contribute 8 cycles.

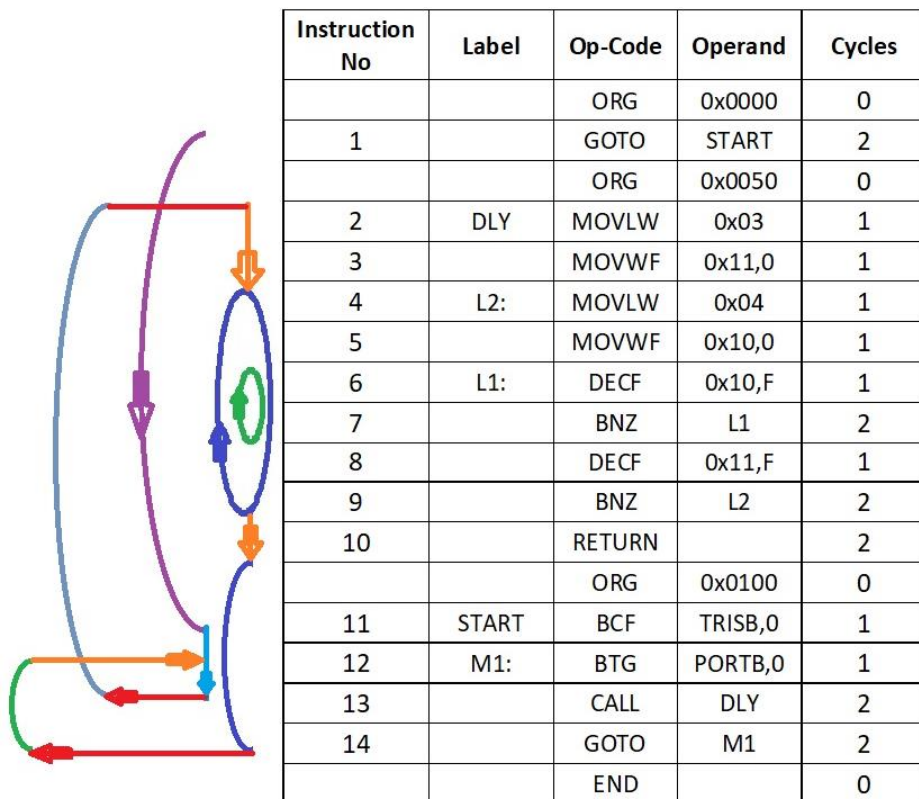


Fig P9.2a: Program demonstrating cycles used.

The number of cycles used for ON Period of Square wave is $2 + DLY + 2 + 1 = 60$ cycles.

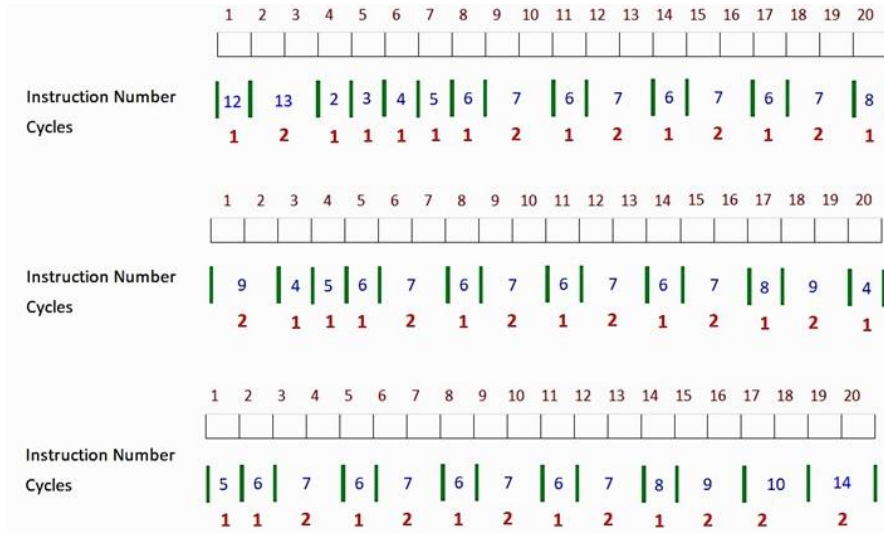


Fig P9.2b: Instruction wise execution & timing of the pulse width.

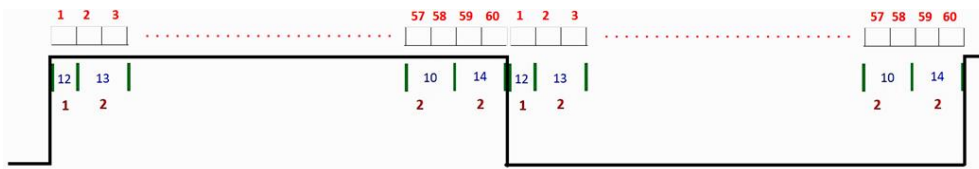


Fig P9.2c: Generated Square waveform & instruction timing cycles.

i.e. $2 \text{ (13)} + \text{DLY} + 2 \text{ (14)} + 1 \text{ (12)}$

Similarly, the number of cycles used for OFF Period of Square wave is $2 + \text{DLY} + 2 + 1 = 60$ cycles.

i.e. $2 \text{ (13)} + \text{DLY} + 2 \text{ (14)} + 1 \text{ (12)}$

The time-period of the square wave generated by this program is the sum of ON + OFF cycles, i.e. 120 cycles. For a 4MHz crystal oscillator 1 Instruction cycle is $1\mu\text{s}$. Hence the time-period is $120\mu\text{s}$ with $60\mu\text{s}$ ON time and $60\mu\text{s}$ OFF time.

This example used loop parameters x and y as 4 and 3 respectively. When they are taken as 1 and 1 the number of delay cycles is 28. So, the minimum pulse width that could be created is 33 cycles with a total of 66 cycles for the square wave period.

A width of less than 66 cycles is not achievable using this program. The effect of program instruction cycles may be considered negligible only when x and y count is large. Additionally, the microcontroller is kept busy executing these instructions during which the microcontroller cannot perform any other task.

9.2 Timer structures & its programming

The use of a hardware timer module overcomes the limitations of design based on software instructions. The delay module section could be implemented by an actual hardware module instead of software instructions. Additionally, the hardware module works in parallel to the CPU. Hence the number of cycles required could be reduced considerably and microcontroller would be freed from executing the software instructions when used with interrupts. This paves the way for reduced number of instructions to be executed by the CPU. Additionally, CPU could be assigned with a different execution task. Hence parallelism is achieved and higher number of tasks could be executed in a specific time-period. Higher the number of parallel modules, higher is the performance.

Table 9.1: Summary of Timers & Features

Device	No of Timers	Timer	Timer		Counter		Edge Select	Pre scale	Post scale
			8 Bit	16Bit	8 Bit	16 Bit			
PIC18F 458/ PIC18 C801	4	Timer 0	Y	Y	Y	Y	Y	Y	-
		Timer 1	-	Y	-	Y	-	Y	-
		Timer 2	Y	-	-	-	-	Y	Y
		Timer 3	-	Y	-	Y	-	Y	-
PIC18F 8720	5	Timer 0	Y	Y	Y	Y	Y	Y	-
		Timer 1	-	Y	-	Y	-	Y	-
		Timer 2	Y	-	-	-	-	Y	Y
		Timer 3	-	Y	-	Y	-	Y	-
		Timer 4	Y	-	-	-	-	Y	Y

Table 9.2: Registers related to Timers.

Device	Timer	Associated Registers				Optional – Interrupt / Clock, Oscillator Related Registers			
PIC18F 458 / PIC18 C801	T0	T0CON	TMR0L	TMR0H	INTCON	-	INTCON2	-	TRISA
	T1	T1CON	TMR1L	TMR1H	PIR1	PIE1	IPR1	INT CON	TRISC
	T2	T2CON	TMR2	PR2					-
	T3	T3CON	TMR3L	TMR3H	PIR2	PIE2	IPR2		TRISC
PIC18F 8720	T0	T0CON	TMR0L	TMR0H	INTCON	-	-	-	TRISA
	T1	T1CON	TMR1L	TMR1H	PIR1	PIE1	IPR1	INT CON	TRISC
	T2	T2CON	TMR2	PR2					-
	T3	T3CON	TMR3L	TMR3H	PIR2	PIE2	IPR2		TRISC
	T4	T4CON	TMR4	PR4	PIR3	PIE3	IPR3	-	-

The following section discusses the number of timers present in the different devices – PIC18F458, PIC18C801 & PIC18F8720 along with their respective structures. A summary of the number of timer's presents and the number of bits available, detection and division features is presented in table 9.1.

From table 9.1 it can be seen that the PIC18F458, PIC18C801 has 4 timers each (Timer 0, 1, 2 & 3), whereas PIC18F8720 has 5 timers (Timer 4 in addition to the timers present in the other devices) (i.e. Timer 0, 1, 2, 3 & 4). All the 4 timers could be configured to work separately with each using a separate control register, flags etc.

Table 9.3: Timers associated Control, Flag, Interrupt bits & related registers.

Device	Timer	Control ON/OFF		Flag		Optional – Interrupt Related			
		Bit	Reg	Bit	Reg	Interrupt Enable		Interrupt Priority	
						Bit	Reg	Bit	Reg
PIC18F458 / PIC18C801	T0	TMR0ON	T0CON	TMR0IF	INTCON	TMR0IE	INTCON	TMR0IP	INTCON2
	T1	TMR1ON	T1CON	TMR1IF	PIR1	TMR1IE	PIE1	TMR1IP	IPR1
	T2	TMR2ON	T2CON	TMR2IF		TMR2IE		TMR2IP	
	T3	TMR3ON	T3CON	TMR3IF	PIR2	TMR3IE	PIE2	TMR3IP	IPR2
PIC18F8720	T0	TMR0ON	T0CON	TMR0IF	INTCON	TMR0IE	INTCON	-	-
	T1	TMR1ON	T1CON	TMR1IF	PIR1	TMR1IE	PIE1	TMR1IP	IPR1
	T2	TMR2ON	T2CON	TMR2IF		TMR2IE		TMR2IP	
	T3	TMR3ON	T3CON	TMR3IF	PIR2	TMR3IE	PIE2	TMR3IP	IPR2
	T4	TMR4ON	T4CON	TMR4IF	PIR3	TMR4IE	PIE3	TMR4IP	IPR3

Table 9.4: Timer Clock/Oscillator Input Pins & Usage

Device	Timer	I/P Pin	Clock In Name	Oscillator		Use for			
						Capture	Compare	PWM	SSP
PIC18F458 / PIC18C801	T0	RA4	TOCKI	-	-	-	-	-	-
	T1	RC0	T1CKI	RC0/T1OSO	RC1/T1OSI	Y	Y	-	-
	T2	-	-	-	-	-	-	Y	Y
	T3	RC0	T1CKI	RC0/T1OSO	RC1/T1OSI	Y	Y	-	-
PIC18F8720	T0	RA4	TOCKI	-	-	-	-	-	-
	T1	RC0	T13CKI	RC0/T1OSO	RC1/T1OSI	Y	Y	-	-
	T2	-	-	-	-	-	-	Y	Y
	T3	RC0	T1CKI	RC0/T1OSO	RC1/T1OSI	Y	Y	-	-
	T4	-	-	-	-	-	-	Y	-

All the timers could be configured as timers whereas Timer 0, Timer 1 and Timer 3 only could be configured as counters. The Timer 2 and Timer 4 are used exclusively as timers. A summary of registers associated with timers is presented in table 9.2. The respective control, flag interrupt bits and related registers is presented in table 9.3. A typical structure of Timer interrupts is shown in Fig 9.1.

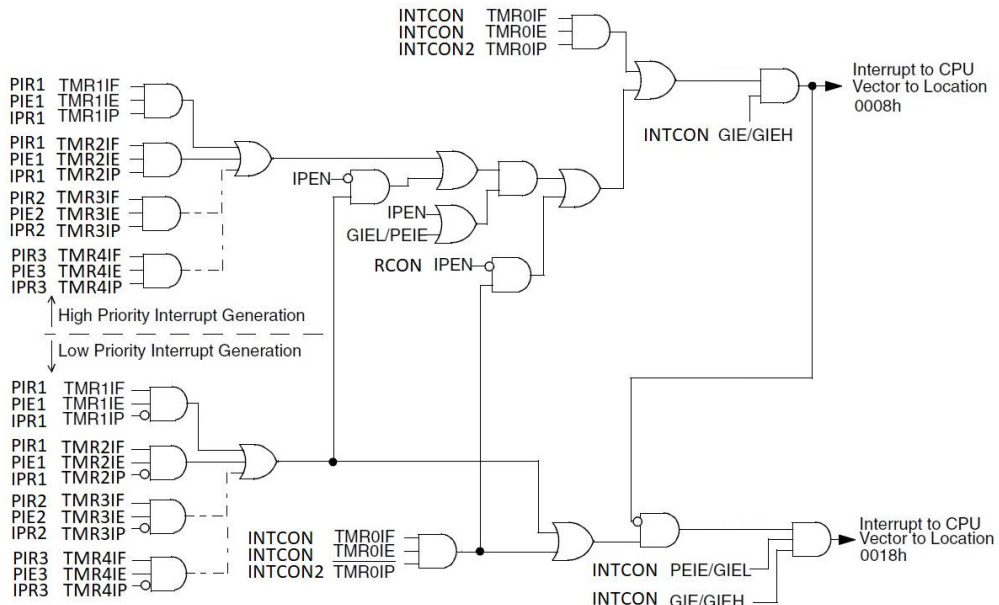


Fig 9.1 Typical structure of Timer Interrupts

The PIC18 devices timers could be configured to select from different clock sources. They are

- Internal clock source that is derived from external main oscillator. (It is the source for the microcontroller CPU instruction execution)
- External clock pins T0CKI or T1CKI/T13CKI pin for the respective timers
- Timer 1's external oscillator connected across T1OSI and T1OSO

Table 9.5: Instruction Cycle Time for different Oscillators frequencies

Crystal Frequency	Instruction Cycle Time $1 / (f_{Osc} / 4)$	Crystal Frequency	Instruction Cycle Time $1 / (f_{Osc} / 4)$
1MHz	4μs	16MHz	0.25 μs
4MHz	1 μs	20MHz	0.2 μs
5.0688MHz	0.7891 μs	22MHz	0.1818 μs
10MHz	0.4 μs	25MHz	0.16 μs
11.0592MHz	0.3617 μs	33MHz	0.1212 μs
12MHz	0.3333 μs	40MHz	0.1 μs

A summary of timer clock/oscillator inputs pins and usage is presented in Table 9.4. The timers are used to control the CCP modules for capture, compare, PWM generation and SSP modules for I2C and SPI communications. It also presents the port pin used for clock input – CLKI. As can be seen from the table Timer 2 and Timer 4 could only be sourced from internal clock signal, Timer 0 could be sourced from internal clock or T0CKI/RA4 pin and Timer 1 and timer 3 could be sourced from internal clock or Timer1 oscillator - T1CKI/T13CKI or RC0/ T1OSO and RC1/ T1OSI respectively.

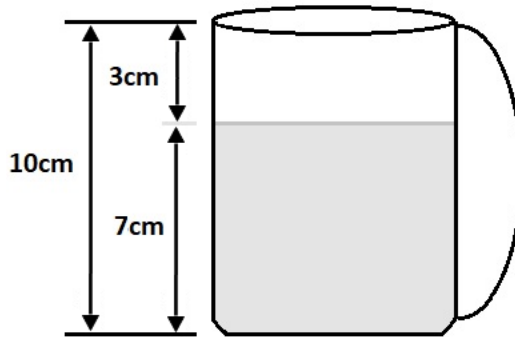


Fig 9.2 Cup Analogy to demonstrate Timer value setting

Calculation of Timer Cycles:

The timer stage of Timer0, Timer1 and Timer3 could be represented as a cup as in Fig 9.2. Here the initial amount liquid (7cm) that is present is equivalent to the preset value that needs to be placed in the timers. The empty area of cup (3cm) represents the number of pulses required for the timer to reach the final value. i.e. to fill the cup to the full ceiling requires

$$7\text{cm (preset)} + 3\text{cm (empty)} = 10\text{cm}$$

To make the cup overflow, liquid of around 1cm additionally need to be poured in. This additional 1cm creates the overflow. So, overflow occurs when total liquid is

$$7\text{cm (preset)} + [3\text{cm (empty)} + 1\text{cm (overflow)}] \text{ (Count)} = 11\text{cm}.$$

So, to make the cup overflow for 4cm liquid (count), the cup needs to be filled with 7cm. The general expression for timer with ceiling T_M , required count T_C , timer preset T_P with an overflow of 1 is given as

$$T_P = T_M - T_C + 1 \quad (9.1)$$

$$\text{Or} \quad T_C = T_M - T_P + 1 \quad (9.2)$$

The preset value is what is stored in timer TMR registers. For a 16-Bit Timer the ceiling value T_M is 65535d or FFFFh and for an 8-Bit Timer the ceiling value T_M is 255d or FFh.

PIC18 is an 8-Bit Microcontroller and hence the registers are accessible 8-Bit wise. Hence in 16-Bit Timer TMR register takes the form TMRH and TMRL where TMRH contains the high byte and TMRL contain the low byte. In an 8-Bit mode the TMR register is 8-bit.

For 16-Bit mode:

For a count of 10d pulses, the preset value is

$$T_p = 65535d - 10d + 1 = 65536d - 10d = 65526d = FFF6h.$$

Therefore TMRH = FFh, & TMRL = F6h

For a preset value of FFF4h (65524), i.e TMRH = FFh, & TMRL = F6h, the count value is

$$TC = 65535d - 65524d + 1 = 65536d - 65524d = 12d = 0Ch$$

For 8-Bit mode:

For a count of 10d (0Ah) pulses, the preset value is

$$T_p = 255d - 10d + 1 = 256d - 10d = 246d = F6h.$$

Therefore TMR = F6h

For a preset value of F4h (244d), i.e TMR = F4h, the count value is

$$TC = 255d - 244d + 1 = 256d - 244d = 12d = 0Ch$$

Conversion of given number of cycles to time

As discussed earlier, for a 16-Bit Timer the ceiling value T_M is 65535d or FFFFh and for an 8-Bit Timer the ceiling value T_M is 255d or FFh. So, the maximum time and number of cycles achievable in 16-Bit mode is higher than that of the 8-bit mode. The achievable time for minimum and maximum number of cycles in 8-Bit and 16-Bit mode for different clock cycles is presented in Table 9.6.

In 16-Bit mode the minimum number of timer cycles achievable is 1 and it is achieved when TMRH = FFh and TMRL = FFh. The maximum number of timer cycles achievable is 65536 and it is achieved when TMRH = 0 and TMRL = 0.

In 8-Bit mode the minimum number of timer cycles achievable is 1 and it is achieved when TMR = FFh. The maximum number of timer cycles achievable is 256 and it is achieved when TMR = 0.

For a count of 10d pulses, the preset value is

$$T_p = 65535d - 10d + 1 = 65536d - 10d = 65526d = FFF6h.$$

Therefore TMRH = FFh, & TMRL = F6h

From the table 9.6 the cycle time is $1\mu\text{s}$ for 4MHz and $0.25\mu\text{s}$ for 16MHz. A count of 10d pulses for a crystal oscillator frequency of 4MHz provides a time of $10 \times 1\mu\text{s} = 10\mu\text{s}$ and at 16MHz provides a time of $10 \times 0.25\mu\text{s} = 2.5\mu\text{s}$.

A general expression can be given as

$$\text{Time } t = \text{Count} \times \text{Cycle Time Period}$$

This value is same for an 8-Bit mode or 16-Bit mode as frequency remains the same, and variation exist only in the range of pulse count and time provided by 8-bit and 16-bit mode.

Table 9.6: Instruction Cycle Time for different Oscillators frequencies (ideal)

Crystal Frequency	Instruction/Timer Cycle Time, μs $1/(f_{\text{Osc}}/4)$	8-Bit Mode (Count + 1)		16-Bit Mode (Count + 1)	
		Min 1d	Max 255d	Min 1d	Max 65536d
		TMR = FFh	TMR = 0h	TMR = FFFFh	TMR = 0h
1MHz	$4\mu\text{s}$	$4\mu\text{s}$	$1024\mu\text{s}$	$4\mu\text{s}$	262.14 ms
4MHz	$1\mu\text{s}$	$1\mu\text{s}$	$256\mu\text{s}$	$1\mu\text{s}$	65.54 ms
5.0688MHz	$0.79\mu\text{s}$	$0.79\mu\text{s}$	$202.01\mu\text{s}$	$0.79\mu\text{s}$	51.71 ms
10MHz	$0.4\mu\text{s}$	$0.4\mu\text{s}$	$102.4\mu\text{s}$	$0.4\mu\text{s}$	26.21 ms
11.0592MHz	$0.36\mu\text{s}$	$0.36\mu\text{s}$	$92.6\mu\text{s}$	$0.36\mu\text{s}$	23.7 ms
12MHz	$0.33\mu\text{s}$	$0.33\mu\text{s}$	$85.32\mu\text{s}$	$0.33\mu\text{s}$	21.84 ms
16MHz	$0.25\mu\text{s}$	$0.25\mu\text{s}$	$64\mu\text{s}$	$0.25\mu\text{s}$	16.38 ms
20MHz	$0.2\mu\text{s}$	$0.2\mu\text{s}$	$51.2\mu\text{s}$	$0.2\mu\text{s}$	13.11 ms
22MHz	$0.18\mu\text{s}$	$0.18\mu\text{s}$	$46.54\mu\text{s}$	$0.18\mu\text{s}$	11.91 ms
25MHz	$0.16\mu\text{s}$	$0.16\mu\text{s}$	$40.96\mu\text{s}$	$0.16\mu\text{s}$	10.49 ms
33MHz	$0.12\mu\text{s}$	$0.12\mu\text{s}$	$31.03\mu\text{s}$	$0.12\mu\text{s}$	7.94 ms
40MHz	$0.1\mu\text{s}$	$0.1\mu\text{s}$	$25.6\mu\text{s}$	$0.1\mu\text{s}$	6.55 ms

Conversion from given time to number of required cycles and timer value

Typically, in real world duty cycle, time-period, ON or OFF times are specified, required instead of instruction cycles because they vary based on the crystal oscillator and type of microcontroller used. Based on the given time and processor, the required number of cycles are calculated, and appropriate values are chosen accordingly. The minimum and maximum time achieved through timer in 8-Bit mode and 16-Bit mode is presented in table 9.6. A time beyond the minimum and maximum shown cannot be achieved normally using one full timer cycle range. Hence for achieving higher times, multiple iterations of full timer cycle range may be used. This is shown through a counter example in program 9.5.

For the required time of $24\mu\text{s}$, the value that needs to be stored in timer registers is calculated as follows.

From the table 9.6 for a 4MHz crystal oscillator the cycle time-period is 1 μ s and for a 16MHz crystal oscillator the cycle time-period is 0.25 μ s.

For a 4MHz crystal oscillator

The number of required cycles = count = $24 \mu\text{s} / 1 \mu\text{s} = 24\text{d} = 18\text{h}$

For 16-Bit mode: For a count of 24d (18h) pulses, the preset value is

$$T_p = 65535\text{d} - 24\text{d} + 1 = 65536\text{d} - 24\text{d} = 65512\text{d} = \text{FFE8h}.$$

Therefore TMRH = FFh, & TMRL = E8h

For 8-Bit mode: For a count of 24d (18h) pulses, the preset value is

$$T_p = 255\text{d} - 24\text{d} + 1 = 256\text{d} - 24\text{d} = 232\text{d} = \text{E8h}.$$

Therefore TMR = E8h

For 16MHz crystal oscillator

The number of required cycles = count = $24 \mu\text{s} / 0.25 \mu\text{s} = 96\text{d} = 60\text{h}$.

For 16-Bit mode: For a count of 96d (60h) pulses, the preset value is

$$T_p = 65535\text{d} - 96\text{d} + 1 = 65536\text{d} - 96\text{d} = 65440\text{d} = \text{FFA0h}.$$

Therefore TMRH = FFh, & TMRL = A0h

For 8-Bit mode: For a count of 96d (60h) pulses, the preset value is

$$T_p = 255\text{d} - 96\text{d} + 1 = 256\text{d} - 96\text{d} = 160\text{d} = \text{A0h}.$$

Therefore TMR = A0h

For a required frequency of 20KHz, with 50% duty cycle at 16MHz, the value that needs to be stored in timer registers is calculated as follows.

From the table 9.6 for a 16MHz crystal oscillator the cycle time-period is 0.25 μ s. A frequency of 20KHz has a time-period of $1/(20\text{KHz}) = 0.05\text{msec}$.

For a 50% duty cycle, ON time = OFF time = $(0.05/2)\text{ms} = 0.025\text{ms} = 25 \mu\text{s}$.

A 25 μ s time is calculated as follows.

The number of required cycles = count = $25 \mu\text{s} / 0.25 \mu\text{s} = 100\text{d} = 64\text{h}$.

For 16-Bit mode: For a count of 100d (64h) pulses, the preset value is

$$T_p = 65535\text{d} - 100\text{d} + 1 = 65536\text{d} - 100\text{d} = 65436\text{d} = \text{FF9Ch}.$$

Therefore TMRH = FFh, & TMRL = 9Ch

For 8-Bit mode: For a count of 100d (64h) pulses, the preset value is

$$T_p = 255\text{d} - 100\text{d} + 1 = 256\text{d} - 100\text{d} = 156\text{d} = 9\text{Ch}.$$

Therefore TMR = 9Ch

For a required frequency of 20KHz, with 60% duty cycle at 16MHz, the value that needs to be stored in timer registers is calculated as follows.

From the table 9.6 for a 16MHz crystal oscillator the cycle time-period is $0.25\ \mu\text{s}$. A frequency of 20KHz has a time-period of $1/(20\text{KHz}) = 0.05\text{msec}$.

The ON time for duty cycle of 60% with a time-period of 0.05msec is
 $0.6 \times 0.05\text{msec} = 0.03\text{msec}$ i.e. $30\ \mu\text{s}$

The OFF time for duty cycle of 60% with a time-period of 0.05msec is
 $0.4 \times 0.05\text{msec} = 0.02\text{msec}$ i.e. $20\ \mu\text{s}$.

A $30\ \mu\text{s}$ ON time is calculated as follows.

The number of required cycles = count = $30\ \mu\text{s} / 0.25\ \mu\text{s} = 120\text{d} = 78\text{h}$.

For 16-Bit mode: For a count of 120d (78h) pulses, the preset value is

$$T_p = 65535\text{d} - 120\text{d} + 1 = 65536\text{d} - 120\text{d} = 65416\text{d} = \text{FF88h}.$$

Therefore TMRH = FFh, & TMRL = 88h

For 8-Bit mode: For a count of 120d (78h) pulses, the preset value is

$$T_p = 255\text{d} - 120\text{d} + 1 = 256\text{d} - 120\text{d} = 136\text{d} = 88\text{h}.$$

Therefore TMR = 88h

A $20\ \mu\text{s}$ OFF time is calculated as follows.

The number of required cycles = count = $20\ \mu\text{s} / 0.25\ \mu\text{s} = 80\text{d} = 50\text{h}$.

For 16-Bit mode: For a count of 80d (50h) pulses, the preset value is

$$T_p = 65535\text{d} - 80\text{d} + 1 = 65536\text{d} - 80\text{d} = 65456\text{d} = \text{FFB0h}.$$

Therefore TMRH = FFh, & TMRL = B0h

For 8-Bit mode: For a count of 80d (50h) pulses, the preset value is

$$T_p = 255\text{d} - 80\text{d} + 1 = 256\text{d} - 80\text{d} = 176\text{d} = \text{B0h}.$$

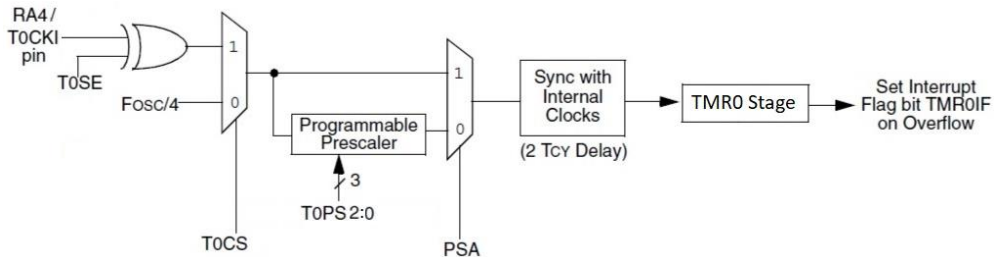
Therefore TMR = B0h

9.2.1 Timer 0

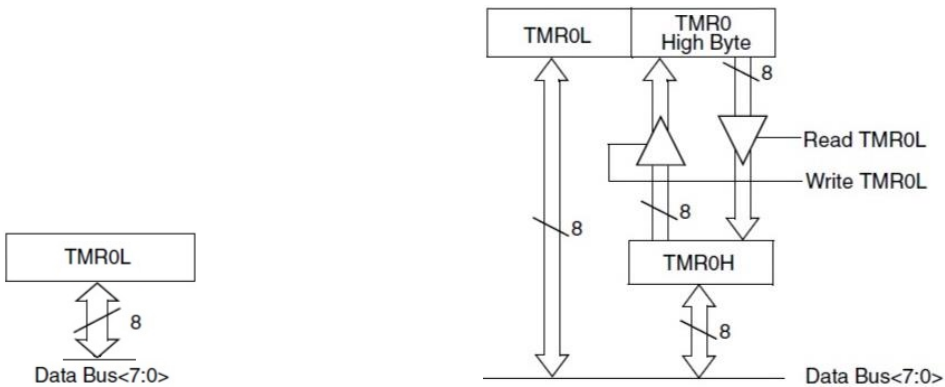
The Timer 0 could be operated as a timer or as a counter. It could be set in 8-bit or a 16-bit mode. It makes use of two timer registers TMR0L & TMR0H and a control register – TOCON. The TMR0L & TMR0H registers hold the timer value. The TOCON register provides different control bits that allows for the configuration and control of the Timer0. These three registers are readable and writeable. The block diagram of Timer 0 is shown in fig 9.3. The control bit details of the same are presented in TOCON control register format.

The T08BIT bit in TOCON register is used to configure the Timer 0 in 8-bit or 16-bit mode. The clock for the Timer0 could be selected from internal instruction clock $F_{\text{Osc}}/4$ or from an external RA4 pin – T0CKI. This selection could be configured through the T0CS bit.

The timer/counter could be made to trigger based on positive edge or negative edge using the T0SE bit. The prescale feature provides the division factors of 2, 4, 8, 16, 32, 64, 128 and 256 that makes timer to work similar to a Mod-8, Mod-16 counter etc. that counts “Mod factor times the oscillator pulses”. The prescale is enabled using the Timer0 Prescale assignment – PSA bit and the actual prescale value is set using the Timer0 prescale select bits – T0PS2:T0PS0.



a) Typical Block diagram of Timer0



b) TMR0 stage in 8-Bit mode

c) TMR0 stage in 16-Bit mode

Fig 9.3 Block diagram of Timer 0

T0CON: Timer0 Control Register Format

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
TMR0ON	T08BIT	T0CS	T0SE	PSA	T0PS2	T0PS1	T0PS0

bit 7

bit 0

Bit No	Bit	Description
bit 7	TMR0ON	Timer0 On/Off Control bit 1 = Enables Timer0 0 = Stops Timer0
bit 6	T08BIT	Timer0 8-bit/16-bit Control bit 1 = Timer0 is configured as an 8-bit timer/counter 0 = Timer0 is configured as a 16-bit timer/counter

bit 5	T0CS	Timer0 Clock Source Select bit 1 = Transition on T0CKI pin 0 = Internal instruction cycle clock (CLKO)
bit 4	T0SE	Timer0 Source Edge Select bit 1 = Increment on high-to-low transition on T0CKI pin 0 = Increment on low-to-high transition on T0CKI pin
bit 3	PSA	Timer0 Prescaler Assignment bit 1 = Timer0 prescaler is not assigned. Timer0 clock input bypasses prescaler. 0 = Timer0 prescaler is assigned. Timer0 clock input comes from prescaler output.
bit 2-0	T0PS2:T0PS0	Timer0 Prescaler Select bits 111 = 1:256 Prescale value 110 = 1:128 Prescale value 101 = 1:64 Prescale value 100 = 1:32 Prescale value 011 = 1:16 Prescale value 010 = 1:8 Prescale value 001 = 1:4 Prescale value 000 = 1:2 Prescale value

Reading and Writing Timer Registers – TMR0L & TMR0H

The timer0 as 8-bit timer/counter is shown in Fig 9.3b. In this mode the timer0 stage comprises of only an 8-bit register TMR0L. Hence it works as an 8-bit timer/counter with 8-bit read/write operations.

The timer0 as 16-Bit timer/counter is shown in Fig 9.3c. In this mode the Timer0 high byte is buffered and cannot be accessed directly for a direct read/write operation.

During a write operation, the TMR0H register must be written first and write to TMR0L register must be performed later to copy the value of TMR0H register into the TMR0H High byte. Similarly, during a read operation, the TMR0L register must be read first to copy value of TMR0H High byte into TMR0H register after which a read of TMR0H register must be performed.

If a TMR0L value is written first, the earlier value that was present in TMR0H would be copied to TMR0 high byte and any value written afterwards to TMR0H is ignored and only applied to TMR0H high byte after another TMR0L write. This is because the TMR0H register is copied into TMR0 high byte upon a write into TMR0L. Hence while writing into Timer 0 in 16-Bit mode, TMR0H register must be written first before writing to TMR0L register.

Similarly, TMR0 High Byte cannot be accessed directly for a read operation. If a TMR0H value is read first, the value that is obtained is not the correct value of TMR0 High Byte in timer. A TMR0L read value would not be in sync with the TMR0H value but a second read of TMR0H after a first TMR0L read produces better result. This is because the TMR0

High Byte is copied into TMR0H register upon a read of TMR0L. Hence while reading Timer 0 in 16-Bit mode the TMR0L register must be read first before reading the TMR0H register.

The advantage of buffering is that it provides the ability to hold all 16 bits of Timer0 that are in sync through the TMR0H & TMR0L. If this buffering was not present, care must be taken to determine whether the read of high byte and low byte are valid. The invalidity occurs when a rollover occurs between a read of high byte and low byte. i.e. Assume at an instant when timer value is 0x2FFF, a first read of high byte produces 0x2F and by the time the low byte is read the count would have incremented making low byte as 0x00 and high byte as 0x30. The actual read value indicates 0x2F00 instead of 0x3000. This doesn't occur when a buffering is present and order is followed, since reading TMR0L places the actual TMR0 high byte immediately into TMR0H so that TMR0L read produces 0xFF and TMR0H read later produces 0x2F.

Timing of Timer

The timer is started or stopped using TMR0ON bit of the TOCON register. Setting the TMR0ON bit to 1 starts the timer whereas clearing to 0 stops the timer.

When a TMR0L register is written, the increment of timer count is inhibited for the following two instruction cycles. Hence for accurate timing the user must store the adjusted value into TMR0 registers.

Additionally, each instruction takes a predefined number of cycles and a corresponding set of instructions of the program takes certain number of cycles. For an accurate timing the user must also take this into account and adjust the value stored into timers TMR0 registers. This is demonstrated using an example in Program 9.8.

Timer Overflow, Flag & Interrupt

In 8-Bit mode, overflow occurs when the timer register - TMR0L changes from 0xFF to 0x00. In 16-Bit mode, overflow occurs when the timer value changes from 0xFFFF to 0x0000. When an overflow occurs the timer overflow flag TMR0IF is set.

An interrupt is generated when TMR0IF is set when the Timer Interrupt Enable - TMR0IE bit is set along with Global interrupt enable - GIE bit.

The TMR0IF, TMR0IE & GIE bits are configured through the INTCON register. The TMR0IE when set to 1 enables the timer0 interrupt and disables when it is set to 0. The GIE bit must be set to 1 to globally enable all the interrupts. The Timer 0 interrupt can be configured to high priority by setting the TMR0IP bit in the INTCON2 register. Timer0 interrupt is set to low priority when the TMR0IP bit is cleared to 0.

Registers related to Timer 0.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR0L	Timer0 Module Low Byte Register							

TMR0H	Timer0 Module High Byte Register							
T0CON	TMR0ON	T08BIT	T0CS	T0SE	PSA	T0PS2	T0PS1	T0PS0
TRISA	—	PORTA Data Direction Register						
INTCON	GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF

Similar to other interrupts, the Timer 0 flag i.e. TMR0IF bit must be cleared using software instructions as part of the Timer0's Interrupt Service Routine - ISR before the interrupt is re-enabled.

Steps to program Timer 0 in 8-Bit & 16-Bit mode

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 0 control - T0CON register with a value to configure timer0 in required 8-Bit/16-Bit mode, select the respective clock input and type of edge clock with the prescale (if required) and stop the timer by clearing TMR0ON bit.
3. If the required duty cycle and frequency is achievable in the required mode,
 - In 8-bit mode load the TMR0L register with appropriate preset/initial count value.
 - In 16-Bit mode load the TMR0H register first and then the TMR0L register with appropriate preset/initial count value.

This step is optional if the first cycle is neglected. A general method of calculating preset value is presented in the earlier section.

4. Clear the timer0 overflow flag - TMR0IF in INTCON register
5. Set the required PORT bit to the required value for generating square wave. (optional)
6. Start the timer0 by setting the TMR0ON bit to 1 in T0CON register.
7. Monitor/Wait for the timer0 overflow flag - TMR0IF in INTCON register to be set.
8. If the timer0 overflow flag - TMR0IF in INTCON register is set, stop the timer0 by clearing TMR0ON bit in T0CON register to 0.
9. Clear the TMR0IF flag in INTCON register
10. Repeat the process by going back to step 3.

Program 9.3: Generate a square wave using Timer 0 in 16-bit mode for 10 pulses.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0
	MOVLW	0x08	; WREG = 0x08, 0x48 for Timer0 in 8-Bit mode

	MOVWF	TOCON	; TOCON = WREG = 0x08, Timer0 in 16-Bit ; mode, internal clock with +ve edge ; No Prescale
HERE	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR0H	; TMR0H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR0L	; TMR0L = WREG = 0xF6, to do total 10 pulses
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BTG	PORTB,1	; Toggle PORTB.1
	BSF	TOCON,TMR0ON	; Start the Timer0
AGAIN	BTFSS	INTCON,TMR0IF	; Check if TMR0IF is set. Skip next instruction ; if set.
	BRA	AGAIN	; Branch to Again, when TMR0IF is not set.
	BCF	TOCON,TMR0ON	; Stop the Timer0
	BRA	HERE	; Branch to Here
	END		

Here the timer is configured in 16-Bit mode made to overflow after 10 pulses. Hence the timer registers TMR0H and TMR0L are loaded with 0xFF and 0xF6 respectively.

The timer count varies with every pulse as follows: 0xFFFF6, 0xFFFF7, 0xFFFF8, 0xFFFF9, 0xFFFFA, 0xFFFFB, 0xFFFFC, 0xFFFFD, 0xFFFFE, 0xFFFFF and overflows to 0x0000 setting the timer overflow flag TMR0IF making a count of 10 pulses.

For an 8-Bit mode the TOCON register must be loaded with 0x48 instead of 0x08.

Program 9.4: Generate a square wave using Timer 0 in 16-bit mode for 10 pulses (50% duty cycle) using modular approach.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR0 module
MTR0	BSF	TOCON,TMR0ON	; Start the Timer0
AGAIN	BTFSS	INTCON,TMR0IF	; Check if TMR0IF is set. Skip next instruction ; if set.
	BRA	AGAIN	; Branch to Again, when TMR0IF is not set.
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BCF	TOCON,TMR0ON	; Stop the Timer0
	RETURN		; Return from MTR0 Module
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0

	MOVLW	0x08	; WREG = 0x08, 0x48 for Timer0 in 8-Bit mode
	MOVWF	T0CON	; T0CON = WREG = 0x08, Timer in 16-Bit mode, internal clock, +ve edge, No Prescale
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
HERE	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR0H	; TMR0H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR0L	; TMR0L = WREG = 0xF6, to do 10 pulses
	BTG	PORTB,1	; Toggle PORTB.1
	CALL	MTR0	; Call MTR0 module
	BRA	HERE	; Branch to Here
	END		

Program 9.5: Generate a square wave using Timer 0 in 16-bit mode with 62.5% Duty Cycle.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR0 module
MTR0	BSF	T0CON,TMR0ON	; Start the Timer0
AGAIN	BTFS	INTCON,TMR0IF	; Check if TMR0IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR0IF is not set.
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BCF	T0CON,TMR0ON	; Stop the Timer0
	RETURN		; Return from MTR0 Module
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0
	MOVLW	0x08	; WREG = 0x08, 0x48 for Timer0 in 8-Bit mode
	MOVWF	T0CON	; T0CON = WREG = 0x08, Timer in 16-Bit mode, internal clock, +ve edge, No Prescale
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
HERE	MOVLW	0x60	; WREG = 0x60, ON Time High Preset value
	MOVWF	TMR0H	; TMR0H = WREG = 0x60
	MOVLW	0x00	; WREG = 0x00, ON Time Low Preset value
	MOVWF	TMR0L	; TMR0L = WREG = 0x00
	BSF	PORTB,1	; Set PORTB.1
	CALL	MTR0	; Call MTR0 module
	MOVLW	0xA0	; WREG = 0xA0, OFF Time High Preset value

	MOVWF	TMR0H	; TMR0H = WREG = 0xA0
	MOVLW	0x00	; WREG = 0x00, OFF Time Low Preset value
	MOVWF	TMR0L	; TMR0L = WREG = 0x00
	BCF	PORTB,1	; Reset PORTB.1
	CALL	MTR0	; Call MTR0 module
	BRA	HERE	; Branch to Here
	END		

Program 9.6: Generate a 100 KHz square wave using Timer 0 with 50% duty cycle with 20MHz crystal oscillator frequency.

A frequency of 100KHz has a time-period of $1/(100\text{KHz}) = 0.01\text{msec} = 10 \mu\text{s}$. From the table 9.6 the cycle time is $0.2 \mu\text{s}$ for 20MHz.

For a required frequency of 100 KHz, with 50% duty cycle at 20MHz, the value that needs to be stored in timer registers is calculated as follows.

For a 50% duty cycle, ON time = OFF time = $(0.01/2) \text{ ms} = 0.005\text{ms} = 5 \mu\text{s}$.

A $5 \mu\text{s}$ time is calculated as follows.

The number of required cycles = count = $5 \mu\text{s} / 0.2 \mu\text{s} = 25\text{d} = 19\text{h}$.

For 16-Bit mode: For a count of 25d (19h) pulses, the preset value is

$$T_p = 65535\text{d} - 25\text{d} + 1 = 65536\text{d} - 25\text{d} = 65511\text{d} = \text{FFE7h}.$$

Therefore, TMR0H = FFh, & TMR0L = E7h

For 8-Bit mode: For a count of 25d (19h) pulses, the preset value is

$$T_p = 255\text{d} - 25\text{d} + 1 = 256\text{d} - 25\text{d} = 231\text{d} = \text{E7h}.$$

Therefore, TMR0L = E7h

Replace the respective TMR0H & TMR0L values in program 9.4 for 16-Bit mode and load the T0CON register with 0x48 instead of 0x08 for an 8-Bit mode.

Program 9.7: Generate a 100 KHz square wave using Timer 0 with 75% duty cycle (7.5 μsec of ON time and 2.5 μsec of OFF time) for a 40MHz crystal oscillator frequency.

A frequency of 100 KHz has a time-period of $1/(100\text{KHz}) = 0.01\text{msec} = 10 \mu\text{s}$. From the table 9.6 the cycle time is $0.1 \mu\text{s}$ for 40MHz.

For a required frequency of 100 KHz, with 75% duty cycle (7.5 μsec of ON time and 2.5 μsec of OFF time) at 40MHz, the value that needs to be stored in timer registers is calculated as follows.

For a 75% duty cycle, ON time = $0.75 * 10 \mu\text{s} = 7.5 \mu\text{s}$ and OFF time = $0.25 * 10 \mu\text{s} = 2.5 \mu\text{s}$.

A $7.5 \mu\text{s}$ ON time is calculated as follows.

The number of required cycles = count = $7.5 \mu\text{s} / 0.1 \mu\text{s} = 75\text{d} = 4\text{Bh}$.

For 16-Bit mode: For a count of 75d (4Bh) pulses, the preset value is

$$T_p = 65535d - 75d + 1 = 65536d - 75d = 65461d = \text{FFB5h}.$$

Therefore, ON Time High Preset value TMR0H = FFh,

ON Time Low Preset value TMR0L = B5h

For 8-Bit mode: For a count of 75d (4Bh) pulses, the preset value is

$$T_p = 255d - 75d + 1 = 256d - 75d = 181d = \text{B5h}.$$

Therefore, ON time preset value TMR0L = B5h

A 2.5 μ s OFF time is calculated as follows.

The number of required cycles = count = $2.5 \mu\text{s} / 0.1 \mu\text{s} = 25d = 19h$.

For 16-Bit mode: For a count of 25d (19h) pulses, the preset value is

$$T_p = 65535d - 25d + 1 = 65536d - 25d = 65511d = \text{FFE7h}.$$

Therefore, OFF Time High Preset value TMR0H = FFh,

OFF Time Low Preset value TMR0L = E7h

For 8-Bit mode: For a count of 25d (19h) pulses, the preset value is

$$T_p = 255d - 25d + 1 = 256d - 25d = 231d = \text{E7h}.$$

Therefore, OFF time preset value TMR0L = E7h

Replace the respective ON Time High/Low Preset value and OFF Time High/Low Preset values into TMR0H & TMR0L values in program 9.5 for 16-Bit mode and load the ON/OFF time preset value into TMR0L and TOCON register with 0x48 instead of 0x08 for an 8-Bit mode.

The programs 9.3 to 9.6 consider only time of the pulses counted by the timer and the time taken for execution of the instructions is neglected. The following program example demonstrates timer example considering also the time taken for execution of instructions.

Program 9.8: Generate a 2 KHz square wave using Timer 0 with 50% duty cycle with 25MHz crystal oscillator frequency considering the instruction cycle time.

Instruc tion No	Label	Op-Code	Operand	Instruc tion Cycles
		ORG	0x00	-
1		GOTO	START	2
		ORG	0x100	-
2	MTR0	BSF	TOCON,TMR0ON	1
3	AGAIN	BTFSS	INTCON,TMR0IF	1 (2)
4		BRA	AGAIN	2
5		BCF	INTCON,TMR0IF	1

6		BCF	TOCON,TMR0ON	1
7		RETURN		2
		ORG	0x0200	-
8	START	BCF	TRISB,1	1
9		MOVLW	0x08	1
10		MOVWF	TOCON	1
11		BCF	INTCON,TMR0IF	1
12	HERE	MOVLW	0xF9	1
13		MOVWF	TMR0H	1
14		MOVLW	0xE6	1
15		MOVWF	TMR0L	1
16		BTG	PORTB,1	1
17		CALL	MTR0	2
18		BRA	HERE	2
		END		-

A frequency of 2KHz has a time-period of $1/(2\text{KHz}) = 0.5\text{msec}$. From the table 9.6 the cycle time is $0.16\text{ }\mu\text{s}$ for 25MHz.

For a required frequency of 2 KHz, with 50% duty cycle at 25MHz, the value that needs to be stored in timer registers is calculated as follows.

For a 50% duty cycle, ON time = OFF time = $(0.5/2)\text{ ms} = 0.25\text{ms}$.

A 25 ms time is calculated as follows.

The number of required cycles = count = $.25\text{ms}/0.16\text{ }\mu\text{s} = 1562.5\text{d} \approx 1562\text{d} = 61\text{Ah}$.

Since this value is more than 256 it cannot be achieved by timer in 8-bit mode.

For 16-Bit mode: For a count of 1562d (61Ah) pulses, the preset value is

$$T_p = 65535\text{d} - 1562\text{d} + 1 = 65536\text{d} - 1562\text{d} = 63974\text{d} = \text{F9E6h}.$$

Therefore, TMR0H = F9h, & TMR0L = E6h

Replace the respective TMR0H & TMR0L registers i.e. values in instruction number 12 and 14 with the above calculated values.

This program makes use of timer and subroutine that is called in the main program to create the required delay. The ideal execution cycles for the delay are the number of pulses counted by the timer multiplied by the time-period of the timer.

$$T_{\text{ideal}} = x * T = (1562\text{d (ON)} + 1562\text{d (OFF)}) * 0.16\text{ }\mu\text{s} = 499.84\text{ }\mu\text{s} \approx 500\text{ }\mu\text{s}.$$

The number of cycles that are taken varies from the ideal execution cycles.

The split for one full program iteration is given as

$$T_{\text{practical}} = 2 + [1*9 + 2 + \text{MTR0} + 2] = 15 + \text{MTR0}$$

$$\text{i.e. } 2 \text{ (1)} + 1 \text{ (8 to 16)} * 9 + 2 \text{ (17)} + \text{TDLY} + 2 \text{ (18)}.$$

The delay created by MTR0 Timer module is given as

$$1 + \{1562d + 2 + 1 + 1 + 2\} - 1 = 1568 \text{ cycles}$$

$$\text{i.e. } 1 \text{ (2)} + \{ \text{(3)(4)} 1562d + 2 \text{ (3)} + 1 \text{ (5)} + 1 \text{ (6)} + 2 \text{ (7)} \} - 1 \text{ (2)}$$

Here instruction number 3 the BTFSS instruction takes 1 cycle when PC is not changed (Flag not set) and takes 2 cycles when PC is changed (Flag set). Till the 1562d pulses are counted by the timer the instruction 3 takes 1 cycle and when the flag is set the same instruction must be executed during which it takes 2 cycles for its completion.

Additionally, the timer gets started during the execution of instruction number 2 hence for calculating the actual count +1 is added and later at the end -1 is done to compensate it in the above expression.

This brings the number of execution cycles for a single iteration of full program to 1583 cycles. The MTR0 module part i.e. instruction 2 to 7 contributes 1568 cycles and the other instructions 1, 8 to 18 contribute 15 cycles.

The number of cycles used for ON Period (i.e. = OFF Period) of square wave is

$$2 + \text{MTR0} + 2 + 1 + 1 + 1 + 1 + 1 + 1 = 9 + \text{MTR0} = 1577 \text{ cycles.}$$

$$\text{i.e. } 2 \text{ (17)} + \text{MTR0} + 2 \text{ (18)} + 1 \text{ (12)} + 1 \text{ (13)} + 1 \text{ (14)} + 1 \text{ (15)} + 1 \text{ (16)}$$

The time-period of the square wave generated by this program is the sum of ON + OFF cycles, i.e. $1577 + 1577 = 3154$ cycles. For 25MHz crystal oscillator 1 Instruction cycle is $0.16\mu\text{s}$. Hence the time-period is $504.64 \mu\text{s}$ ($3154 * 0.16\mu\text{s}$) with $252.32 \mu\text{s}$ ($1577*0.16\mu\text{s}$) ON time and $252.32 \mu\text{s}$ ($1577*0.16\mu\text{s}$) OFF time.

So, for an ideal required $500 \mu\text{s}$ (i.e. 1562 cycles), the practical achieved time is $504.64\mu\text{s}$ (i.e. 3154 cycles), producing an error of $4.64 \mu\text{s}$ (i.e. 30 cycles) of additional time due to instructions. Hence the timer preset value is to be modified to compensate for this additional time. So instead of making the timer to count 1562d pulses by loading F9E6h into TMR0H & TMR0L, an equivalent preset value that allows for timer count of 1547d pulses is loaded into the Timer registers TMR0H & TMR0L.

$$\text{i.e. } 65535 - 1547 + 1 = 63989d = \text{F9F5h}$$

This implies that TMR0H must be loaded with F9h & TMR0L with F5h.

When the TMR0H & TMR0L are loaded with FFFFh the delay created by MTR0 Timer module is given as

$$1 + \{1+2 + 2 + 1 + 1 + 2\} - 1 = 9 \text{ cycles}$$

$$\text{i.e. } 1 \text{ (2)} + \{1 \text{ (3)} + 2 \text{ (4)} + 2 \text{ (3)} + 1 \text{ (5)} + 1 \text{ (6)} + 2 \text{ (7)}\} - 1 \text{ (2)}$$

Hence the total number of cycles used for ON Period (i.e. = OFF Period) of square wave is

$$2 + \text{MTR0} + 2 + 1 + 1 + 1 + 1 + 1 = 9 + 9 = 18 \text{ cycles}$$

The time period of the square wave generated by this program is the sum of ON + OFF cycles, i.e. $18 + 18 = 36$ cycles.

Hence a width of less than 36 cycles ($36 \times 0.16 \mu\text{s}$) i.e. less than $5.76 \mu\text{s}$ is not achievable by this program with a 25MHz crystal oscillator even though table 9.6 suggests that $0.16 \mu\text{s}$ is the minimum achievable time. This is because the table 9.6 shows the ideal time.

Similarly, the practical execution time or the number of cycles taken could be calculated for program 9.5 with 62.5% duty cycle or any other program.

Program 9.9: Generate a square wave using Timer 0 in 8-bit mode for 10 pulses (instruction cycles) (50% duty cycle) with a prescale of 2 using modular approach.

The program for this is same as program 9.4 except for the configuration value loaded into TOCON and the preset value is loaded only into TMR0L register since it is now in 8-Bit mode.

Since it need configured in 8-bit mode the T08Bit must be set to 1 and the prescale assignment bit PSA must be cleared and prescale bits TOPS2:0 bits must be cleared to set prescale of 2. Hence the value of 0x40 must be loaded into TOCON register.

Since a prescale of 2 is assigned, a pulse is sent to timer0 stage for every two instruction cycles. Hence to get the same amount of delay equivalent to actual 10 instruction cycles, the count value required is $(10/2)$ and hence TMR0L must be loaded with $255 - 5 + 1 = 251_{10} = 0_{16}\text{FB}$. The complete program is repeated again here with the updated values.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR0 module
MTR0	BSF	TOCON,TMR0ON	; Start the Timer0
AGAIN	BTFSS	INTCON,TMR0IF	; Check if TMR0IF is set. Skip next ; instruction ; if set.
	BRA	AGAIN	; Branch to Again, when TMR0IF is not set.
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BCF	TOCON,TMR0ON	; Stop the Timer0
	RETURN		; Return from MTR0 Module
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0

	MOVLW	0x40	; WREG = 0x40, for Timer0 in 8-Bit mode
	MOVWF	T0CON	; T0CON = WREG = 0x40, Timer in 8-Bit ; mode, internal clock, +ve edge, ; Prescale 2
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
HERE	MOVLW	0xFB	; WREG = 0xFB
	MOVWF	TMR0L	; TMR0L = WREG = 0xFB, to do 10 pulses ; with prescale 2
	BTG	PORTB,1	; Toggle PORTB.1
	CALL	MTR0	; Call MTR0 module
	BRA	HERE	; Branch to Here
	END		

9.2.1.1 Programming Timer0 interrupts

The table 9.3 presents the flag and control bits for using the timer interrupts. The following program 9.10 and program 9.11 presents example programs of timer0 with interrupts. A typical structure of Timer interrupts is shown in Fig 9.1.

Steps to program Timer 0 with interrupts

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 0 control - T0CON register with a value to configure timer0 in required 8-Bit/16-Bit mode, select the respective clock input and type of edge clock with the prescale (if required) and stop the timer by clearing TMR0ON bit.
3. If the required duty cycle and frequency is achievable in the required mode,
 - In 8-bit mode load the TMR0L register with appropriate preset/initial count value.
 - In 16-Bit mode load the TMR0H register first and then the TMR0L register with appropriate preset/initial count value.

This step is optional if the first cycle is neglected. A general method of calculating preset value is presented in the earlier section.

4. Clear the timer0 overflow flag TMR0IF in INTCON register.
5. Enable TMR0IE- Timer0 Interrupt Enable bit in INTCON register for enabling Timer0 interrupt.
6. Set the Timer0 interrupt priority to high or low by setting/clearing TMR0IP bit in INTCON2 register.
7. Enable GIE- Global Interrupt Enable bit in INTCON register for enabling interrupts.
8. Start the timer0 by setting the TMR0ON bit in T0CON register and proceed to do an alternate task.

9. In the respective priority location call/place the timer0's ISR.
10. As part of ISR, if required, check to see if the interrupt is generated due to timer0 overflow by checking whether the TMR0IF in INTCON register is set or not. This is required when multiple interrupts are used. This step is optional if only timer0 interrupt is enabled.
11. If the timer0 overflow flag - TMR0IF is set, the timer may be stopped by clearing TMR0ON bit in TOCON register.
12. Clear the TMR0IF flag in INTCON register.
13. Set the required PORT bit to the required value for generating square wave. (optional)
14. Load the timer registers TMR0H & TMR0L (only TMR0L in 8-Bit mode) with appropriate preset values
15. Start the timer0 again by setting TMR0ON bit in TOCON register before returning from the ISR.
16. Whenever interrupt occurs timer0's ISR is executed automatically. i.e. step 10 to step 15.

Program 9.10: Generate a square wave using Timer 0 in 16-bit mode for 10 pulses using interrupts.

The example program presented here only demonstrate single timer0 interrupt, does not consider multiple interrupts. Hence the program doesn't include timer0 programming step no 11.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	TO_ISR	; Go to Timer0 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer0 ISR
TO_ISR	BTG	PORTB,1	; Toggle PORTB.1
	BCF	TOCON,TMR0ON	; Stop the Timer0
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR0H	; TMR0H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR0L	; TMR0L = WREG = 0xF6, to do 10 pulses
	BSF	TOCON,TMR0ON	; Start the Timer0
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program

START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0
	MOVLW	0x08	; WREG = 0x08, 0x48 for Timer0 in 8-Bit ; mode
	MOVWF	TOCON	; TOCON = WREG = 0x08, Timer in 16-Bit ; mode, internal clock with +ve edge and ; No Prescale
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR0H	; TMR0H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR0L	; TMR0L = WREG = 0xF6, to do 10 pulses
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BSF	INTCON,TMR0IE	; Enable Timer0 Interrupt
	BSF	INTCON2,TMR0IP	; Make Timer 0 High Priority
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	TOCON,TMR0ON	; Start the Timer0
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

Program 9.11: Generate a square wave using Timer 0 in 16-bit mode for 62.5% Duty Cycle using interrupts.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	TO_ISR	; Go to Timer0 ISR
	ORG	0x100	; Set Origin 0x0100 for MTR0 module
TO_ISR	BCF	TOCON,TMR0ON	; Stop the Timer0
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	TSTFSNZ	0x00,0	; Check content of [0x00], if it is 0, ; it indicates end of -ve Level, ; If 0 go to SL1
	BRA	SL1	; Branch to SL1
	MOVLW	0x60	; WREG = 0x60
	MOVWF	TMR0H	; TMR0H = WREG = 0x60
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR0L	; TMR0L = WREG = 0x00
	BSF	PORTB,1	; Set PORTB.1
	BRA	SL2	; BRA SL2
SL1	MOVLW	0xA0	; WREG = 0xA0
	MOVWF	TMR0H	; TMR0H = WREG = 0xA0
	MOVLW	0x00	; WREG = 0x00

	MOVWF	TMR0L	; TMR0L = WREG = 0x00
	BCF	PORTB,1	; Reset PORTB.1
SL2	BSF	TOCON,TMR0ON	; Start the Timer0
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,1	; Set PortB Pin 1 as Output, TRISB.1=0
	MOVLW	0x01	; WREG = 0x01
	MOVWF	0x00,0	; File Reg [0x00] = WREG = 0x01
	MOVLW	0x08	; WREG = 0x08, 0x48 for Timer0 in 8-Bit mode
	MOVWF	TOCON	; TOCON = WREG = 0x08, Timer in 16-Bit mode, internal clock , +ve edge & No Prescale
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	BSF	INTCON,TMR0IE	; Enable Timer0 Interrupt
	BSF	INTCON2,TMR0IP	; Make Timer 0 High Priority
	NOP		; No Operation
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	TOCON,TMR0ON	; Start the Timer0
HERE	NOP		; No operation, Logic of optional alternate function may be placed here.
	BRA	HERE	; Branch to Here
	END		

9.2.2 Timer 1

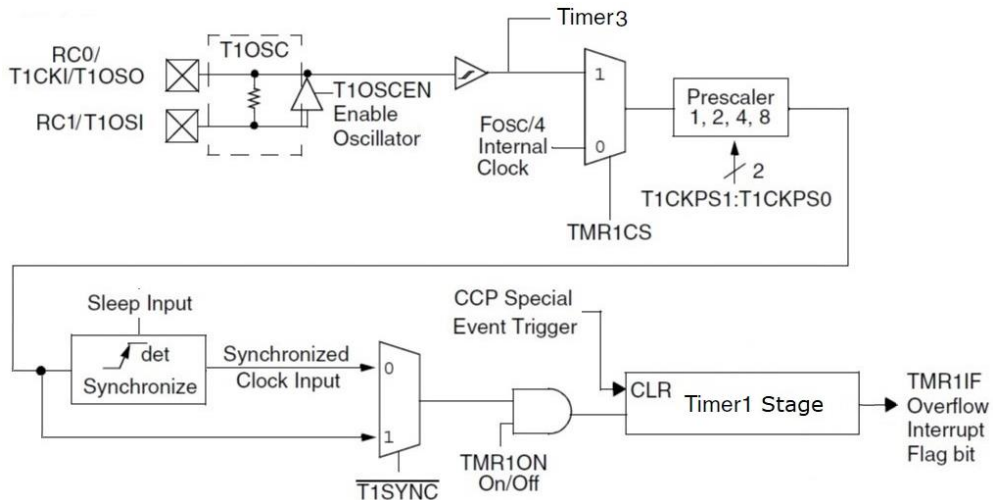
The Timer 1 could be operated as a timer or as a counter. It operates in 16-bit mode. It makes use of two timer registers TMR1L & TMR1H and a control register – T1CON. The TMR1L & TMR1H registers hold the timer value. The TOCON register provides different control/configuration bits that allows for the configuration and control of the Timer1. These three registers are readable and writeable.

The timer registers could be read/written in one 16-Bit operation (buffered) or as two 8-Bit operations (non-buffered). The block diagram of Timer 1 is shown in fig 9.4 and the control bit details of the same is presented in the T1CON control register format.

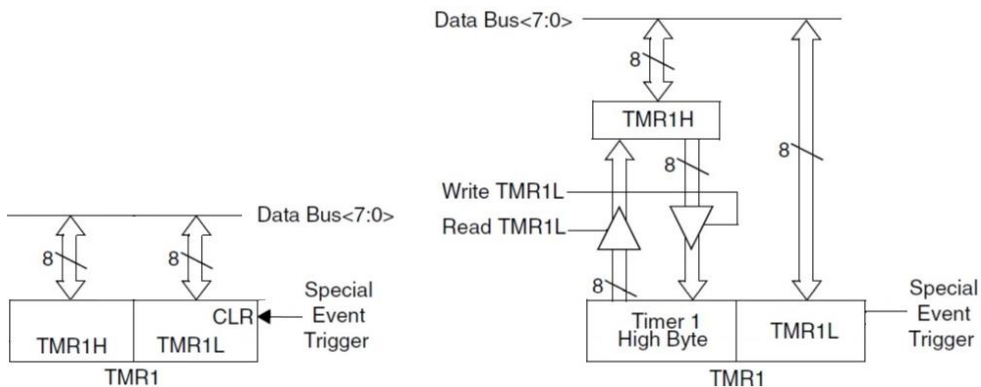
The RD16: 16-bit Read/Write mode Enable bit in T1CON register is used to configure the timer1 for read/write in one 16-bit operation when set to 1 and when set to 0 it is configured for two 8-bit operations. The timer1 is always triggered by the positive/rising edge of the clock and is not configurable like the timer0 for a falling edge. The clock for the Timer1 could be selected from the main internal clock i.e. $F_{osc}/4$ or from an external T1CKI/RC0 pin or an oscillator connected between T1OSO/RC0 and T1OSI/RC1 pins. This selection between internal clock and external clock is configured through the TMR1CS bit.

When set to 0 the internal clock i.e. $F_{Osc}/4$ is applied to timer stage, whereas when set to 1 the external oscillator signal is applied.

The timer1 oscillator connected across the T1OSO/RC0 and T1OSI/RC1 pins could also be used as a source for timer3.



a) Typical block diagram of Timer 1



TMR1 stage in Read/Write mode

b) as two 8-Bit operations

c) as one 16-Bit operation

Fig 9.4 Block diagram of Timer 1

While using an external oscillator between T1OSO/RC0 and T1OSI/RC1 pins, the timer1 oscillator must be enabled using the T1OSCEN Enable Oscillator bit. If the bit is set to 1 the T1 Oscillator is enabled, else it is disabled. When oscillator is enabled, the pins RC0 and RC1 act as inputs and the setting of TRISC<1:0> is neglected. Additionally, the Timer1 need to be configured to synchronize the external clock input. The timer stage may be provided a clock signal that is synchronized or non-synchronized based on the T1SYNC: Timer1 External

Clock Input Synchronization Select bit of T1CON register. The timer1 stage is applied with the synchronized clock signal when T1SYNC bit is set to 0, and a non-synchronized signal is applied when it is set to 1.

The prescale feature is similar to Timer0 and only provides division factors of 1, 2, 4 and 8 to make the timer work similar to a Mod-2, Mod-4 or Mod-8 counter etc. that counts “Mod factor times the oscillator pulses”. The pulses pass through the prescale stage always and hence the Timer1 structure does not contain any prescale enable bit. The actual prescale value is set using the Timer1 prescale select bits – T1CKPS1:T1CKPS0.

T1CON: Timer1 Control Register Format

R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
RD16	—	T1CKPS1	T1CKPS0	T1OSCEN	<u>T1SYNC</u>	TMR1CS	TMR1ON
bit 7		bit 0					

Bit No	Bit	Description
bit 7	RD16	16-bit Read/Write Mode Enable bit 1 = Enables register read/write of Timer1 in one 16-bit operation 0 = Enables register read/write of Timer1 in two 8-bit operations
bit 6	Unimplemented	Read as '0'
bit 5-4	T1CKPS1:T1CKPS0	Timer1 Input Clock Prescale Select bits 11 = 1:8 Prescale value 10 = 1:4 Prescale value 01 = 1:2 Prescale value 00 = 1:1 Prescale value
bit 3	T1OSCEN	Timer1 Oscillator Enable bit 1 = Timer1 oscillator is enabled 0 = Timer1 oscillator is shut-off The oscillator inverter and feedback resistor are turned off to eliminate power drain.
bit 2	<u>T1SYNC</u>	Timer1 External Clock Input Synchronization Select bit When TMR1CS = 1: 1 = Do not synchronize external clock input 0 = Synchronize external clock input When TMR1CS = 0: This bit is ignored. Timer1 uses the internal clock when TMR1CS = 0.
bit 1	TMR1CS	Timer1 Clock Source Select bit 1 = External clock from pin RC0/T1OSO/T1CKI (on the rising edge) 0 = Internal clock (FOSC/4)
bit 0	TMR1ON	Timer1 On bit 1 = Enables Timer1 0 = Stops Timer1

Reading and Writing Timer Registers – TMR1L & TMR1H

The Read/Write of timer registers as two 8-Bit operations is shown in Fig 9.4b. In this mode the two registers are read/written independently and are not buffered. The Read/Write of timer registers as one 16-Bit operation is shown in Fig 9.4c. Timer1 is similar to Timer0 when configured in to read/write of 16-bit in one operation. Timer1 High byte is buffered and cannot be accessed directly for a direct read/write operation.

Similar to Timer0, during a write operation, the TMR1H register must be written first and write to TMR1L register must be performed later so as to copy the value of TMR1H register into the TMR1H High byte. Similarly, during a read operation, the TMR1L register must be read first to copy value of TMR1H High byte into TMR1H register after which a read of TMR1H register must be performed.

The timer registers TMR1H & TMR1L could be reset/cleared to 0 when the CCP/ECCP module is using timer1 and is configured to generate special event trigger.

Timing of Timer

The timer is started or stopped using TMR1ON bit of the T1CON register. Setting the TMR1ON bit to 1 starts the timer whereas clearing to 0 stops the timer.

Every PIC18 instruction takes a predefined number of cycles and a corresponding set of instructions of the program takes certain number of cycles. For an accurate timing the user must take this into account and adjust the value stored into timers TMR1 registers. An equivalent example of the same is demonstrated in program 9.8 for timer0. The same approach could be applied to timer1 for determining the practical execution time.

Timer Overflow, Flag & Interrupt

The timer1 operates always in 16-Bit mode and overflow occurs when the timer value changes from 0xFFFF to 0x0000. When an overflow occurs the timer overflow flag TMR1IF is set.

An interrupt is generated when TMR1IF is set when the Timer Interrupt Enable – TMR1IE bit is set along with Global interrupt enable - GIE bit. Additionally, Peripheral interrupt enable – PEIE bit is required for all the timers except for Timer0.

The TMR1IF bit is accessed through PIR1 register, TMR1IE through the PIE1 register. The TMR1IE when set to 1 enables the timer1 interrupt and disables when it is set to 0. The GIE & PEIE bits are configured through the INTCON register. The GIE bit must be set to 1 to globally enable all the interrupts and PEIE must be set to 1 to enable all the peripheral interrupts. The Timer 1 interrupt can be configured to high priority by setting the TMR1IP bit in the IPR1 register. Timer1 interrupt is set to low priority when the TMR1IP bit is cleared to 0.

Similar to other interrupts, the Timer 1 flag i.e. TMR1IF bit must be cleared using software instructions as part of the Timer1's Interrupt Service Routine - ISR before the interrupt is re-enabled.

Registers related to Timer 1.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register							
TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register							
T1CON	RD16	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON
INTCON	GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
PIR1	PSPIF	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
PIE1	PSPIE	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
IPR1	PSPIP	ADIP	RCIP	TXIP	SSPIP	CCP1IP	TMR2IP	TMR1IP

Steps to program Timer 1

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 1 control – T1CON register with a value to configure timer1 in required 8-Bit RW/16-Bit RW mode, select the respective clock input with the prescale (if required) and stop the timer by clearing TMR1ON bit.
3. If the required duty cycle and frequency is achievable in the required mode, load the TMR1H register first and then the TMR1L register with appropriate preset/initial count value. (Order of loading may not be followed if configured in 8-bit R/W mode). This step is optional if the first cycle is neglected.
A general method of calculating preset value is presented in the earlier section.
4. Clear the timer1 overflow flag – TMR1IF in PIR1 register.
5. Set the required PORT bit to the required value for generating square wave. (optional)
6. Start the timer1 by setting the TMR1ON bit to 1 in T1CON register.
7. Monitor/Wait for the timer1 overflow flag – TMR1IF in PIR1 register to be set.
8. If the timer1 overflow flag – TMR1IF in PIR1 register is set, stop the timer1 by clearing TMR1ON bit in T1CON register to 0.
9. Clear the TMR1IF flag in PIR1 register.
10. Repeat the process by going back to step 3.

Program 9.12: Generate a square wave using Timer 1 for 10 pulses.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
ORG	0x00	; Set Origin 0x0000 of Flash memory	
GOTO	START	; Go to Start of Main program.	

	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T1CON	; T1CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, ; No Synchronization, Prescale 1:1
HERE	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR1L	; TMR1L = WREG = 0xF6, to do 10 pulses
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BTG	PORTB,2	; Toggle PORTB.2
	BSF	T1CON,TMR1ON	; Start the Timer1
AGAIN	BTFSS	PIR1,TMR1IF	; Check if TMR1IF is set. Skip next instruction ; if set.
	BRA	AGAIN	; Branch to Again, when TMR1IF is not set.
	BCF	T1CON,TMR1ON	; Stop the Timer1
	BRA	HERE	; Branch to Here
	END		

Program 9.13: Generate a square wave using Timer 1 in 16-bit R/W mode for 10 pulses (50% duty cycle) using modular approach

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR1 module
MTR1	BSF	T1CON,TMR1ON	; Start the Timer1
AGAIN	BTFSS	PIR1,TMR1IF	; Check if TMR1IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR1IF is not set.
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BCF	T1CON,TMR1ON	; Stop the Timer1
	RETURN		; Return from MTR1 Module
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T1CON	; T1CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, No ; Synchronization, without Prescale
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
HERE	MOVLW	0xFF	; WREG = 0xFF

	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR1L	; TMR1L = WREG = 0xF6, to do 10 pulses
	BTG	PORTB,2	; Toggle PORTB.2
	CALL	MTR1	; Call MTR1 module
	BRA	HERE	; Branch to Here
	END		

Program 9.14: Generate a square wave using Timer 1 in 16-bit R/W mode for 10 pulses (50% duty cycle) using modular approach with timer1's external oscillator and prescale 2.

The program for this is same as program 9.12 except for the configuration value loaded into T1CON and the preset value loaded into TMR1H & TMR1L registers.

Since it must be configured with prescale of 2, the prescale bits T1CKPS1:0 bits must be set to 01, since external T1 oscillator is used the oscillator must be enabled by setting the T1OSCEN bit to 1, synchronized by setting the T1SYNC to 0 and selecting the external oscillator signal instead of internal fosc/4 clock by setting the TMR1CS bit to 1. Hence the value of 0x9A must be loaded into T1CON register.

Since a prescale of 2 is assigned, a pulse is sent to timer1 stage for every two instruction cycles. Hence to get the same amount of delay (assuming same frequency of $f_{osc}/4$) equivalent to actual 10 instruction cycles, the count value required is $(10/2)$ and hence TMR1H & TMR1L registers must be loaded with $65535 - 5 + 1 = 65531d = 0xFFFB$. The complete program is repeated here with the updated values.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR1 module
MTR1	BSF	T1CON,TMR1ON	; Start the Timer1
AGAIN	BTFSS	PIR1,TMR1IF	; Check if TMR1IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR1IF is not set.
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BCF	T1CON,TMR1ON	; Stop the Timer1
	RETURN		; Return from MTR1 Module
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x9A	; WREG = 0x9A
	MOVWF	T1CON	; T1CON = WREG = 0x9A, 16-Bit ; Read/Write Mode, External T1OSC ON,

			; External Clock, With Synchronization, ; Prescale 2.
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
HERE	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xFB	; WREG = 0xFB
	MOVWF	TMR1L	; TMR1L = WREG = 0xFB, to do 10 ; equivalent pulses with prescale 2
	BTG	PORTB,2	; Toggle PORTB.2
	CALL	MTR1	; Call MTR1 module
	BRA	HERE	; Branch to Here
	END		

9.2.2.1 Programming Timer1 interrupts

The table 9.3 presents the flag and control bits for using the timer interrupts. The following program 9.15 and program 9.16 presents example programs of timer1 with interrupts. A typical structure of Timer interrupts is shown in Fig 9.1.

Steps to program Timer 1 with interrupts

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 1 control – T1CON register to configure timer1 in required 8-Bit RW/16-Bit RW mode, select the respective clock input with the prescale (if required) and stop the timer by clearing TMR1ON bit.
3. If the required duty cycle and frequency is achievable, load the TMR1H register first and then the TMR1L register with appropriate preset/initial count value. This step is optional if the first cycle is neglected.

A general method of calculations is presented in the earlier section.

4. Clear the timer1 overflow flag TMR1IF in PIR1 register.
5. Enable TMR1IE- Timer1 Interrupt Enable bit in PIE1 register for enabling Timer1 interrupt.
6. Set the Timer1 interrupt priority to high or low by setting/clearing TMR1IP bit in IPR1 register.
7. Enable GIE- Global Interrupt Enable bit in INTCON register for enabling interrupts.
8. Enable PEIE- Peripheral Interrupt Enable bit in INTCON register for enabling peripheral interrupts. This is required for timer 1, 2, 3 and 4 but is not required for timer1.
9. Start the timer1 by setting the TMR1ON bit in T1CON register and proceed to do an alternate task.

10. In the respective priority location call/place the timer1's ISR.
11. As part of ISR, if required check to see if the interrupt is generated due to timer1 overflow by checking whether the TMR1IF in PIR1 register is set or not. This is required when multiple interrupts are used. This step is optional if only timer1 interrupt is enabled.
12. If the timer1 overflow flag – TMR1IF is set, the timer may be stopped by clearing TMR1ON bit in T1CON register.
13. Clear the TMR1IF flag in PIR1 register.
14. Set the required PORT bit to the required value for generating square wave. (optional)
15. Load the timer registers TMR1H & TMR1L with appropriate preset values
16. Start the timer1 again by setting TMR1ON bit in T1CON register before returning from the ISR.
17. Whenever interrupt occurs timer1's ISR is executed automatically. i.e. step 11 to step 16.

Program 9.15: Generate a square wave using Timer 1 for 10 pulses with interrupt

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T1_ISR	; Go to Timer1 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer0 ISR
T1_ISR	BTG	PORTB,2	; Toggle PORTB.2
	BCF	T1CON,TMR1ON	; Stop the Timer1
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR1L	; TMR1L = WREG = 0xF6, to do 10 pulses
	BSF	T1CON,TMR1ON	; Start the Timer1
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T1CON	; T1CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, ; No Synchronization, Prescale 1:1

	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR1L	; TMR1L = WREG = 0xF6, to do 10 pulses
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BSF	PIE1,TMR1IE	; Enable Timer1 Interrupt
	BSF	IPR1,TMR1IP	; Make Timer 1 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T1CON,TMR1ON	; Start the Timer1
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

Program 9.16: Generate a square wave of 62.5% Duty Cycle using Timer 1 with interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR1 module
MTR1	BSF	T1CON,TMR1ON	; Start the Timer1
AGAIN	BTFSS	PIR1,TMR1IF	; Check if TMR1IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR1IF is not set.
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BCF	T1CON,TMR1ON	; Stop the Timer1
	RETURN		; Return from MTR1 Module
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T1CON	; T1CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, No ; Synchronization, without Prescale
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
HERE	MOVLW	0x60	; WREG = 0x60
	MOVWF	TMR1H	; TMR1H = WREG = 0x60
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1L	; TMR1L = WREG = 0x00
	BSF	PORTB,2	; Set PORTB.2
	CALL	MTR1	; Call MTR1 module

	MOVLW	0xA0	; WREG = 0xA0
	MOVWF	TMR1H	; TMR1H = WREG = 0xA0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1L	; TMR1L = WREG = 0x00
	BCF	PORTB,2	; Reset PORTB.2
	CALL	MTR1	; Call MTR1 module
	BRA	HERE	; Branch to Here
	END		

9.2.3 Timer 2

The Timer 2 is always operated as an 8-bit timer. It makes use of two timer registers TMR2 & PR2 and a control register – T2CON. The TMR2 register acts a preset register similar to TMR0L register of Timer0 and holds the current count value. The PR2 register is called as period register and is used to set the maximum ceiling count/time-period. The T2CON register provides different control/configuration bits that allows for the configuration and control of the Timer2. These three registers are readable and writeable.

The block diagram of Timer 2 is shown in fig 9.5 and the control bit details of the same is presented in the T2CON control register format.

The Timer2 always operate based on main internal clock i.e. $F_{osc}/4$ and hence no control bits are provided to choose between internal or external clock. Similarly, no synchronization is required as the clock used is the internal clock and hence no control bit is provided related to synchronization.

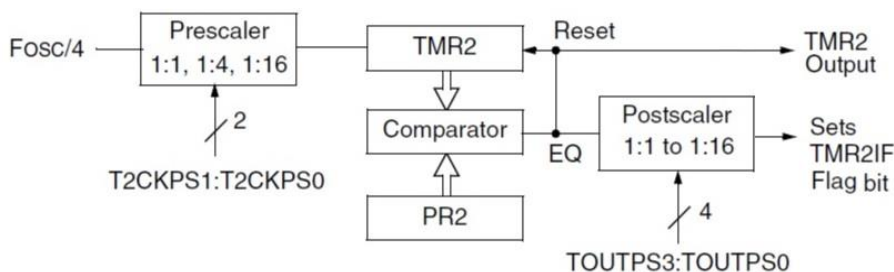


Fig 9.5 Block diagram of Timer 2

Unlike Timer0, Timer1 and Timer3 the Timer2 and Timer4 include postscale select bits along with the prescale select bits. The timer pulses pass through both the prescale and postscale and hence no control bits are provided for enabling the prescale or postscale.

T2CON: Timer2 Control Register Format for PIC18F458 & PIC18C801

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
bit 7							bit 0

Bit No	Bit	Description
bit 7	Unimplemented	Read as '0'
bit 6-3	TOUTPS3:TOUTPS0	Timer2 Output Postscale Select bits 0000 = 1:1 Postscale 0001 = 1:2 Postscale • • • 1111 = 1:16 Postscale
bit 2	TMR2ON	Timer2 On bit 1 = Timer2 is on 0 = Timer2 is off
bit 1-0	T2CKPS1:T2CKPS0	Timer2 Clock Prescale Select bits 00 = Prescaler is 1 01 = Prescaler is 4 1x = Prescaler is 16

T2CON: Timer2 Control Register Format for PIC18F8720

Variation of Bit 6,3 w.r.t PIC18F458 & PIC18C801

Bit No	Bit	Description
bit 6-3	T2OUTPS3:T2OUTPS0	Timer2 Output Postscale Select bits 0000 = 1:1 Postscale 0001 = 1:2 Postscale • • • 1111 = 1:16 Postscale

The prescale feature of time2 is similar to Timer1/Timer3 and only provides division factors of 1, 4 and 16 to make the timer work similar to a Mod-4, or Mod-16 counter etc. that counts “Mod factor times the oscillator pulses”. The actual prescale value is set using the Timer2 clock prescale select bits – T2CKPS1:T2CKPS0.

The additional scaling feature present in timer2 is the postscale feature which is similar to prescale. Here the timer2 output is applied to the postscale stage whereas in a prescale the input clock is applied to the prescale stage. This provides division factors of 1, 2, 3, 4, upto 16 to make the timer work similar to a Mod-N counter etc. that counts “Mod factor times the timer output pulses”. The actual postscale value is set using the Timer2 output postscale select bits – TOUTPS3: TOUTPS0 in PIC18F458 and PIC18C801 and using the T2OUTPS3: T2OUTPS0 in PIC18F8720.

Reading and Writing Timer Registers – TMR2 & PR2

The operation of timer2 is similar to other timers. However, the setting of overflow flag is different from timer0, 1 and 3. Here the PR2 period register holds the ceiling/period

value whereas TMR2 holds the preset/current count value. The number of pulses that are counted by timer2 is equal to the difference between the period register PR2 and the TMR2 preset/counter register. Hence the TMR2 and PR2 registers must be loaded with the respective values based on the required number of pulses.

The timer register TMR2 could be reset/cleared to 0 when the CCP/ECCP module is configured to use timer2 as its source for PWM mode.

Timing of Timer

The timer2 is started or stopped using TMR2ON bit of the T2CON register. Setting the TMR2ON bit to 1 starts the timer whereas clearing to 0 stops the timer.

Every PIC18 instruction takes a predefined number of cycles and a corresponding set of instructions of the program takes certain number of cycles. For an accurate timing the user must take this into account and adjust the value stored into timer2's TMR2 and PR2 registers. An equivalent example of the same is demonstrated in program 9.8 for timer0. The same approach could be applied to timer2 for determining the practical execution time.

Timer Overflow, Flag & Interrupt

The timer2 operates always in 8-Bit mode and flag is set when the timer count value in TMR2 matches the value in period register PR2. This is different to other timers 0, 1 and 3 which set timer flag when TMR value changes from 0xFFFF to 0x0000. When the PR2 and TMR2 match occurs the timer overflow flag TMR2IF is set.

An interrupt is generated when TMR2IF is set when the Timer Interrupt Enable – TMR2IE bit is set along with Global interrupt enable - GIE bit. Additionally, Peripheral interrupt enable – PEIE bit is required for all the timers except for Timer0.

The TMR2IF bit is accessed through PIR1 register, TMR2IE through the PIE1 register. The TMR2IE when set to 1 enables the timer2 interrupt and disables when it is set to 0. The GIE & PEIE bits are configured through the INTCON register. The GIE bit must be set to 1 to globally enable all the interrupts and PEIE must be set to 1 to enable all the peripheral interrupts. The Timer2 interrupt can be configured to high priority by setting the TMR2IP bit in the IPR1 register. The timer2 interrupt is set to low priority when the TMR2IP bit is cleared to 0.

Similar to other interrupts, the Timer2 flag i.e. TMR2IF bit must be cleared using software instructions as part of the Timer2's Interrupt Service Routine - ISR before the interrupt is re-enabled.

Registers related to Timer 2.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PR2	Timer2 Period Register							
TMR2	Timer2 Module Register							

T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
INTCON	GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
PIR1	PSPIF	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
PIE1	PSPIE	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
IPR1	PSPIP	ADIP	RCIP	TXIP	SSPIP	CCP1IP	TMR2IP	TMR1IP

Steps to program Timer 2

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 2 control – T2CON register with a value to configure timer2 to select the respective prescale and postscale (if required) and stop the timer by clearing TMR2ON bit.
3. If the required duty cycle and frequency is achievable in the required mode, load the TMR2 register with preset value and PR2 register with appropriate ceiling/period value. This step is optional if the first cycle is neglected.
4. Clear the timer2 overflow flag – TMR2IF in PIR1 register.
5. Set the required PORT bit to the required value for generating square wave. (optional)
6. Start the timer2 by setting the TMR2ON bit in T2CON register.
7. Monitor/Wait for the timer2 overflow flag – TMR2IF to be set in PIR1 register.
8. If the timer2 overflow flag – TMR2IF in PIR1 register is set, stop the timer2 by clearing the TMR2ON bit in T2CON register to 0.
9. Clear the TMR2IF flag in T2CON register.
10. Repeat the process by going back to step 3.

Relationship between Prescale, postscale and timer values:

A general expression for the relation between total required cycles (T_{req}), prescale (C_{Pre}), postscale (C_{Post}) and timer pulse count value is expressed as

$$T_{req} = C_{Pre} * (PR2 - TMR2) * C_{Post}$$

When TMR2 is 0 it can be simplified to

$$T_{req} = C_{Pre} * PR2 * C_{Post}$$

Program 9.17: Generate a square wave using Timer 2 for 12 pulses.

Since timer2 works based on comparison of TMR2 and PR2 registers values, also it is different from other timers – Timer 0, 1 and 3 the values that must be loaded into the TMR2 and PR2 is presented here.

For 12 pulses with no prescale and post scale with difference between PR2 and TMR2 as 12. Hence If the TMR2 is loaded with 0x00 the PR2 may be loaded with 0x0C. If the TMR2 is loaded with 0xE3 the PR2 must be loaded with 0xEF.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,3	; Set PortB Pin 3 as Output, TRISB.3=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, No Prescale, ; Postscale
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR2	; PR2 = WREG = 0x0C, to do total 12 pulses
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BTG	PORTB,3	; Toggle PORTB.3
	BSF	T2CON,TMR2ON	; Start the Timer2
AGAIN	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR2IF is not set.
	BCF	T2CON,TMR2ON	; Stop the Timer2
	BRA	HERE	; Branch to Here
	NOP		; No Operation
	END		

Program 9.18: Generate a square wave using Timer 2 for 12 pulses (50% duty cycle) using modular approach.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR2 module
MTR2	BSF	T2CON,TMR2ON	; Start the Timer2
AGAIN	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR2IF is not set.
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BCF	T2CON,TMR2ON	; Stop the Timer2
	RETURN		; Return from MTR2 Module

	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,3	; Set PortB Pin 3 as Output, TRISB.3=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, No Prescale, ; Postscale
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR2	; PR2 = WREG = 0x0C, to do 12 pulses
	BTG	PORTB,3	; Toggle PORTB.3
	CALL	MTR2	; Call MTR2 module
	BRA	HERE	; Branch to Here
	END		

Program 9.19: Generate a square wave using Timer 2 for 60 pulses (50% duty cycle) using modular approach with prescale of 4 and postscale of 3.

The program for this is same as program 9.18 except for the configuration value loaded into T2CON and the preset value loaded into TMR2 & PR2 registers.

Since it must be configured with prescale of 4, the prescale bits T2CKPS1:0 bits must be set to 01 and since the postscale of 3 is required the postscale select bits TOUTPS3:0 must be set to 0010. Hence the value of 0x11 must be loaded into T2CON register.

Since a prescale of 4 is assigned, a pulse is sent to timer2 stage for every four instruction cycles. Also, an additional postscale of 3 is assigned. Hence to get the required delay equivalent to 60 instruction cycles, the count value required is $60 / (4 * 3)$ i.e. 5 pulse count. Hence TMR2 register may be loaded with 0x00, and PR2 register must be loaded with 5 instead of $(255 - 5 + 1)$ i.e. 251d or 0xFB in 8-bit mode or $(65535 - 5 + 1)$ 65531d or 0xFFFFB in 16-bit mode. Timers 0, 1, and 3 are loaded with value of 0xFC or 0xFFFFC whereas the Timer2 and Timer 4 are loaded with value of 0x05.

So, for due to prescale for every 4 instruction cycles timer would get one pulse. For five timer pulses the TMR2 matches PR2 and generates TMR2 Output. Due to a postscale of 3, the TMR2IF flag is set for three TMR2 Output pulses. So total $4 * 5 * 3 = 60$ cycles.

The complete program is repeated here with the updated values. Note that the calculations presented here are done under ideal conditions and the instruction execution cycle time is neglected.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory

	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR2 module
MTR2	BSF	T2CON,TMR2ON	; Start the Timer2
AGAIN	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR2IF is not set.
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BCF	T2CON,TMR2ON	; Stop the Timer2
	RETURN		; Return from MTR2 Module
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,3	; Set PortB Pin 3 as Output, TRISB.3=0
	MOVLW	0x11	; WREG = 0x11
	MOVWF	T2CON	; T2CON = WREG = 0x11, Prescale 4, ; Postscale 3.
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x05	; WREG = 0x05
	MOVWF	PR2	; PR2 = WREG = 0x05, to do 60 pulses
	BTG	PORTB,3	; Toggle PORTB.3
	CALL	MTR2	; Call MTR2 module
	BRA	HERE	; Branch to Here
	END		

Program 9.20: Generate a square wave using Timer 2 with crystal oscillator frequency of 20MHz with 50% duty cycle of 12 μ sec using modular approach.

From table 9.6, a 20MHz crystal provides an instruction cycle time of 0.2 μ sec. Hence the total required number of pulses = 12 μ sec/0.2 μ sec = 60 pulses.

We have earlier seen that for 60 pulses count may be achieved by timer2 when CPre = 4, CPost = 3, with TMR2 = 0, and PR2 = 5. Hence the above program of 9.19 satisfies the given requirement.

Program 9.21: Generate a 62.5% duty cycle square wave with Timer 2.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for MTR2 module

MTR2	BSF	T2CON,TMR2ON	; Start the Timer2
AGAIN	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR2IF is not set.
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BCF	T2CON,TMR2ON	; Stop the Timer2
	RETURN		; Return from MTR2 Module
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,3	; Set PortB Pin 3 as Output, TRISB.3=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, No Prescale, ; Postscale
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0xA0	; WREG = 0xA0
	MOVWF	PR2	; PR2 = WREG = 0xA0
	BSF	PORTB,3	; Set PORTB.3
	CALL	MTR2	; Call MTR2 module
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x60	; WREG = 0x60
	MOVWF	PR2	; PR2 = WREG = 0x60
	BCF	PORTB,3	; Reset PORTB.3
	CALL	MTR2	; Call MTR2 module
	BRA	HERE	; Branch to Here
	END		

9.2.3.1 Programming Timer2 interrupts

The table 9.3 presents the flag and control bits for using the timer interrupts. The following program 9.22 presents an example program of timer2 with interrupts. A typical structure of Timer interrupts is shown in Fig 9.1.

Steps to program Timer 2 with interrupts

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 2 control – T2CON register to configure timer2 to select the prescale and post scale (if required) and stop the timer by clearing TMR2ON bit.

3. If the required duty cycle and frequency is achievable, load the TMR2 register with a preset value and PR2 register with a ceiling/period value. This step is optional if the first cycle is neglected.
4. Clear the timer2 overflow flag TMR2IF in PIR1 register.
5. Enable TMR2IE- Timer2 Interrupt Enable bit in PIE1 register for enabling Timer2 interrupt.
6. Set the Timer2 interrupt priority to high or low by setting/clearing TMR2IP bit in IPR1 register.
7. Enable GIE- Global Interrupt Enable bit in INTCON register for enabling interrupts.
8. Enable PEIE- Peripheral Interrupt Enable bit in INTCON register for enabling peripheral interrupts. This is required for timer 1, 2, 3 and 4 but is not required for timer2.
9. Start the timer2 by setting the TMR2ON bit in T2CON register and proceed to do an alternate task.
10. In the respective priority location call/place the timer2's ISR.
11. As part of ISR, if required check to see if the interrupt is generated due to timer2 overflow by checking whether the TMR2IF in PIR1 register is set or not. This is required when multiple interrupts are used. This step is optional if only timer2 interrupt is enabled.
12. If the timer2 overflow flag – TMR2IF is set, the timer may be stopped by clearing TMR2ON bit in T2CON register.
13. Clear the TMR2IF flag in PIR1 register.
14. Set the required PORT bit to the required value for generating square wave. (optional)
15. Load the timer registers TMR2 & PR2 with appropriate preset values
16. Start the timer1 again by setting TMR2ON bit in T2CON register before returning from the ISR.
17. Whenever interrupt occurs timer2s's ISR is executed automatically. i.e. step 11 to step 16.

Program 9.22: Generate a square wave using Timer 2 for 12 pulses (50% duty cycle) using modular approach and interrupts

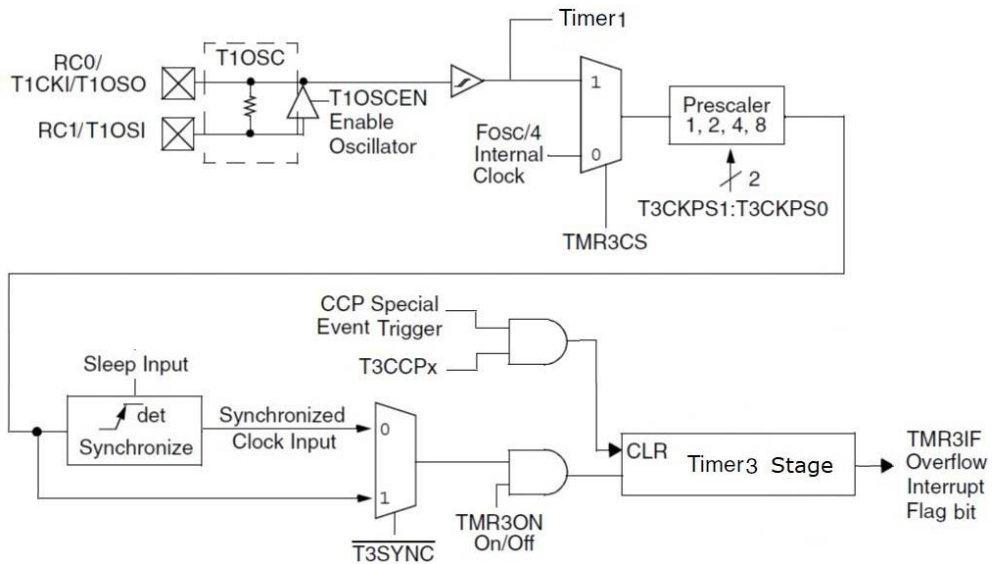
	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T2_ISR	; Go to Timer2 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer2 ISR

T2_ISR	BCF	T2CON,TMR2ON	; Stop the Timer2
	BTG	PORTB,3	; Toggle PORTB.3
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR2	; PR2 = WREG = 0x0C, to do total 12 pulses
	BSF	T2CON,TMR2ON	; Start the Timer2
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,3	; Set PortB Pin 3 as Output, TRISB.3=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, ; No Prescale, Postscale
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR2	; TMR2 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR2	; PR2 = WREG = 0x0C, to do total 12 pulses
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BSF	PIE1,TMR2IE	; Enable Timer2 Interrupt
	BSF	IPR1,TMR2IP	; Make Timer 2 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T2CON,TMR2ON	; Start the Timer2
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

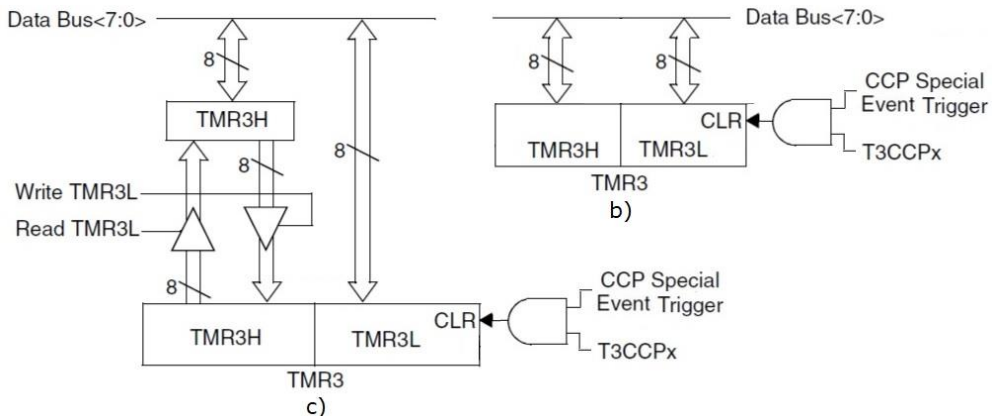
9.2.4 Timer 3

The Timer 3 is similar to Timer 1 and could be operated as a 16-bit timer or as a 16-bit counter. It makes use of two timer registers TMR3L & TMR3H and a control register – T3CON. The TMR3L & TMR3H registers hold the timer value. The T3CON register provides different control bits that allows for the configuration and control of the Timer3. These three registers are readable and writeable.

The timer registers could be read/written in one 16-Bit operation (buffered) or as two 8-Bit operations (non buffered). The block diagram of Timer 3 is shown in fig 9.6 and the control bit details of the same is presented in the T3CON control register format.



a) Typical block diagram of Timer 3



b) TMR3 stage in 8-Bit Read/Write mode c) TMR3 stage in 16-Bit Read/Write mode

Fig 9.6 Block diagram of Timer 3

The RD16: 16-bit Read/Write mode Enable bit in T3CON register is used to configure the timer3 for read/write in one 16-bit operation when set to 1 and when set to 0 it is configured for two 8-bit operations. Similar to timer 1 the timer3 is always triggered by the positive/rising edge (after first falling edge) of the clock and is not configurable like the timer0 for a falling edge. The clock for the Timer3 could be selected similar to timer 1 from the main internal clock i.e. $F_{osc}/4$ or from an external T1CKI/RC0 pin or an oscillator connected between T1OSO/RC0 and T1OSI/RC1 pins.

T3CON: Timer3 Control Register Format - PIC18F458

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
RD16	T3ECCP1	T3CKPS1	T3CKPS0	T3CCP1	<u>T3SYNC</u>	TMR3CS	TMR3ON

bit 7

bit 0

Bit No	Bit	Description
bit 7	RD16	16-bit Read/Write Mode Enable bit 1 = Enables register read/write of Timer3 in one 16-bit operation 0 = Enables register read/write of Timer3 in two 8-bit operations
bit 6,3	T3ECCP1:T3CCP1	Timer3 and Timer1 to CCP1/ECCP1 Enable bits 1x = Timer3 is the clock source for compare/capture CCP1 and ECCP1 modules 01 = Timer3 is the clock source for compare/capture of ECCP1, Timer1 is the clock source for compare/capture of CCP1 00 = Timer1 is the clock source for compare/capture CCP1 and ECCP1 modules
bit 5-4	T3CKPS1:T3CKPS0	Timer3 Input Clock Prescale Select bits 11 = 1:8 Prescale value 10 = 1:4 Prescale value 01 = 1:2 Prescale value 00 = 1:1 Prescale value
bit 2	<u>T3SYNC</u>	Timer3 External Clock I/P Synchronization Control bit. (Not usable if the system clock comes from Timer1/Timer3.) When TMR3CS = 1: 1 = Do not synchronize external clock input 0 = Synchronize external clock input When TMR3CS = 0: This bit is ignored. Timer3 uses the internal clock when TMR3CS = 0.
bit 1	TMR3CS	Timer3 Clock Source Select bit 1 = External clock input from Timer1 oscillator or T1CKI (on the rising edge after the first falling edge) 0 = Internal clock (FOSC/4)
bit 0	TMR3ON	Timer3 On bit 1 = Enables Timer3 0 = Stops Timer3

T3CON: Timer3 Control Register PIC18C801, Variation of Bit 6,3 w.r.t PIC18F458

Bit No	Bit	Description
bit 6, 3	T3CCP2:T3CCP1	Timer3 and Timer1 to CCPx Enable bits 1x = Timer3 is the clock source for compare/capture CCP modules 01 = Timer3 is the clock source for compare/capture Of CCP2, Timer1 is the clock source for compare/

capture of CCP1
00 = Timer1 is the clock source for compare/capture
CCP modules

T3CON: Timer3 Control Register PIC18F8720, Variation of Bit 6,3 w.r.t PIC18F458

Bit No	Bit	Description
bit 6, 3	T3CCP2:T3CCP1	Timer3 and Timer1 to CCPx Enable bits 11 = Timer3 and Timer4 are the clock sources for CCP1 through CCP5 10 = Timer3 and Timer4 are the clock sources for CCP3 through CCP5; Timer1 and Timer2 are the clock sources for CCP1 and CCP2 01 = Timer3 and Timer4 are the clock sources for CCP2 through CCP5; Timer1 and Timer2 are the clock sources for CCP1 00 = Timer1 and Timer2 are the clock sources for CCP1 through CCP5

This selection between internal clock and external clock is configured through the TMR3CS bit. When set to 0 the internal clock i.e. $F_{osc}/4$ is applied to timer stage, whereas when set to 1 the external oscillator signal is applied. It should be noted that timer3 cannot have an independent crystal oscillator of its own. The oscillator connected between T1OSO/RC0 and T1OSI/RC1 pins acts as the external oscillator for both timer1 and timer3.

While using an external oscillator for timer3 between T1OSO/RC0 and T1OSI/RC1 pins, the timer1 oscillator must be enabled using the T1OSCEN Enable Oscillator bit of the T1CON register. If the bit is set to 1 the Timer1 Oscillator is enabled, else it is disabled. When oscillator is enabled, the pins RC0 and RC1 act as inputs and the setting of TRISC<1:0> is neglected.

Even though the crystal oscillator is controlled by T1CON register of timer1, the Timer3 need to be configured to synchronize the external clock input. The timer3 stage may be provided a clock signal that is synchronized or non-synchronized based on the T3SYNC : Timer3 External Clock Input Synchronization Select bit of T3CON register. The timer3 stage is applied with the synchronized clock signal when T3SYNC bit is set to 0, and a non-synchronized signal is applied when it is set to 1.

The prescale feature is similar to Timer1 and only provides division factors of 1, 2, 4 and 8 to make the timer work similar to a Mod-2, Mod-4 or Mod-8 counter etc. that counts “Mod factor times the oscillator pulses”. The pulses pass through the prescale stage always and hence the Timer3 structure also does not contain any prescale enable bit. The actual prescale value is set using the Timer3 prescale select bits – T3CKPS1:T3CKPS0.

A variation in the bit 6,3 of T3CON register exists among the three PIC18 devices considered here. This variation is due to the number and type of CCP modules present in

the respective devices. The T3CON<6,3> bits are Timer3 and Timer1 to CCPx Enable bits that used to configure/select the corresponding timer source (Timer1 or Timer3). The variation in these two bits is also presented above along with the T3CON register format.

Reading and Writing Timer Registers – TMR3L & TMR3H

The Read/Write of timer registers as two 8-Bit operations is shown in Fig 9.6b. In this mode the two registers are read/written independently and are not buffered. The Read/Write of timer registers as one 16-Bit operation is shown in Fig 9.6c. Timer3 is similar to Timer0 when configured in to read/write of 16-bit in one operation. Timer3 High byte is buffered and cannot be accessed directly for a direct read/write operation.

In one 16-bit operation mode, similar to Timer0 & Timer1, during a write operation, the TMR3H register must be written first and write to TMR3L register must be performed later to copy the value of TMR3H register into the TMR3H High byte. Similarly, during a read operation, the TMR3L register must be read first to copy value of TMR3H High byte into TMR3H register after which a read of TMR3H register must be performed.

The timer registers TMR3H & TMR3L could be reset/cleared to 0 when the CCP/ECCP module is using timer3 and is configured to generate special event trigger.

Timing of Timer

The timer3 is started or stopped using TMR3ON bit of the T3CON register. Setting the TMR3ON bit to 1 starts the timer whereas clearing to 0 stops the timer.

Every PIC18 instruction takes a predefined number of cycles and a corresponding set of instructions of the program takes certain number of cycles. For an accurate timing the user must take this into account and adjust the value stored into timers TMR3 registers. An equivalent example of the same is demonstrated in program 9.8 for timer0. The same approach could be applied to timer3 for determining the practical execution time.

Timer Overflow, Flag & Interrupt

The timer3 operates always in 16-Bit mode and overflow occurs when the timer value changes from 0xFFFF to 0x0000. When an overflow occurs the timer overflow flag TMR3IF is set.

An interrupt is generated when TMR3IF is set when the Timer Interrupt Enable – TMR3IE bit is set along with Global interrupt enable - GIE bit. Additionally, Peripheral interrupt enable – PEIE bit is required for all the timers except for Timer0.

The TMR3IF bit is accessed through PIR2 register, TMR2IE through the PIE2 register. The TMR3IE when set to 1 enables the timer3 interrupt and disables when it is set to 0. The GIE & PEIE bits are configured through the INTCON register. The GIE bit must be set to 1 to globally enable all the interrupts and PEIE must be set to 1 to enable all the peripheral interrupts. The Timer 3 interrupt can be configured to high priority by setting the TMR3IP bit

in the IPR2 register. Timer3 interrupt is set to low priority when the TMR3IP bit is cleared to 0.

Similar to other interrupts, the Timer 3 flag i.e TMR3IF bit must be cleared using software instructions as part of the Timer3's Interrupt Service Routine - ISR before the interrupt is re-enabled.

Registers related to Timer 3.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR3L	Holding Register for the Least Significant Byte of the 16-bit TMR3 Register							
TMR3H	Holding Register for the Most Significant Byte of the 16-bit TMR3 Register							
T1CON	RD16	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON
T3CON	RD16	T3ECCP1	T3CKPS1	T3CKPS0	T3CCP1	T3SYNC	TMR3CS	TMR3ON
INTCON	GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
PIR2	—	CMIF	—	EEIF	BCLIF	LVDIF	TMR3IF	ECCP1IF
PIE2	—	CMIE	—	EEIE	BCLIE	LVDIE	TMR3IE	ECCP1IE
IPR2	—	CMIP	—	EEIP	BCLIP	LVDIP	TMR3IP	ECCP1IP

Steps to program Timer 3

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)
2. Load the timer 3 control – T3CON register with a value to configure timer3 in required 8-Bit RW/16-Bit RW mode, select the respective clock input with the prescale (if required) and stop the timer by clearing TMR3ON bit.
3. Additionally, if Timer1 external oscillator is used, the Timer1 external oscillator must be enabled by setting the T1OSCEN bit of T1CON register.
4. If the required duty cycle and frequency is achievable in the required mode, load the TMR3H register first and then the TMR3L register with appropriate preset/initial count value. (Order of loading may not be followed if configured in 8-bit R/W mode). This step is optional if the first cycle is neglected.

A general method of calculating preset value is presented in the earlier section.

5. Clear the timer3 overflow flag – TMR3IF in PIR2 register.
6. Set the required PORT bit to the required value for generating square wave. (optional)
7. Start the timer3 by setting the TMR3ON bit to 1 in T3CON register.
8. Monitor/Wait for the timer3 overflow flag – TMR3IF in PIR2 register to be set.
9. If the timer3 overflow flag – TMR3IF in PIR2 register is set, stop the timer3 by clearing TMR3ON bit in T3CON register to 0.
10. Clear the TMR3IF flag in PIR2 register.
11. Repeat the process by going back to step 4.

Program 9.23: Generate a square wave using Timer 3 for 10 pulses (50% duty cycle)

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,4	; Set PortB Pin 4 as Output, TRISB.4=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T3CON	; T3CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, ; No Synchronization, Prescale 1:1
HERE	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR3H	; TMR3H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR3L	; TMR3L = WREG = 0xF6, to do 10 pulses
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	BTG	PORTB,4	; Toggle PORTB.4
	BSF	T3CON,TMR3ON	; Start the Timer3
AGAIN	BTFS	PIR2,TMR3IF	; Check if TMR3IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR3IF is not set.
	BCF	T3CON,TMR3ON	; Stop the Timer3
	BRA	HERE	; Branch to Here
	NOP		; No Operation
	END		

Note: If a timer1 external crystal oscillator is used, T1OSCEN bit of T1CON need to be set using the following instruction before/after loading T3CON with 0x84.

i.e. BSF T1CON, T1OSCEN

or loading appropriate value into WREG and moving to T1CON using instruction:

MOVWF T1CON,0

9.2.4.1 Programming Timer3 interrupts

The table 9.3 presents the flag and control bits for using the timer interrupts. The following program 9.24 and program 9.25 presents example programs of timer3 with interrupts. A typical structure of Timer interrupts is shown in Fig 9.1.

Steps to program Timer 3 with interrupts

1. Clear the corresponding TRIS bit for generating square wave. This may be performed later before using the PORT bit. (optional)

2. Load the timer 3 control – T3CON register to configure timer3 in required 8-Bit RW/16-Bit RW mode, select the respective clock input with the prescale (if required) and stop the timer by clearing TMR3ON bit.
3. Additionally, if Timer1 external oscillator is used, the Timer1 external oscillator must be enabled by setting the T1OSCEN bit of T1CON register.
4. If the required duty cycle and frequency is achievable, load the TMR3H register first and then the TMR3L register with appropriate preset/initial count value. This step is optional if the first cycle is neglected.
A general method of calculations is presented in the earlier section.
5. Clear the timer3 overflow flag TMR3IF in PIR2 register.
6. Enable TMR3IE- Timer3 Interrupt Enable bit in PIE2 register for enabling Timer3 interrupt.
7. Set the Timer3 interrupt priority to high or low by setting/clearing TMR3IP bit in IPR2 register.
8. Enable GIE- Global Interrupt Enable bit in INTCON register for enabling interrupts.
9. Enable PEIE- Peripheral Interrupt Enable bit in INTCON register for enabling peripheral interrupts. This is required for timer 1, 2, 3 and 4 but is not required for timer1.
10. Start the timer3 by setting the TMR3ON bit in T3CON register and proceed to do an alternate task.
11. In the respective priority location call/place the timer3's ISR.
12. As part of ISR, if required check to see if the interrupt is generated due to timer3 overflow by checking whether the TMR3IF in PIR2 register is set or not. This is required when multiple interrupts are used. This step is optional if only timer3 interrupt is enabled.
13. If the timer3 overflow flag – TMR3IF is set, the timer may be stopped by clearing TMR3ON bit in T3CON register.
14. Clear the TMR3IF flag in PIR2 register.
15. Set the required PORT bit to the required value for generating square wave. (optional)
16. Load the timer registers TMR3H & TMR3L with appropriate preset values
17. Start the timer3 again by setting TMR3ON bit in T3CON register before returning from the ISR.
18. Whenever interrupt occurs timer3's ISR is executed automatically. i.e. step 12 to step 17.

Program 9.24: Generate a square wave using Timer 3 for 12 pulses (50% duty cycle) with interrupts.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T3_ISR	; Go to Timer3 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer3 ISR
T3_ISR	BTG	PORTB,4	; Toggle PORTB.4
	BCF	T3CON,TMR3ON	; Stop the Timer3
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR3H	; TMR3H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR3L	; TMR3L = WREG = 0xF6, to do 10 pulses
	BSF	T3CON,TMR3ON	; Start the Timer3
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,4	; Set PortB Pin 4 as Output, TRISB.4=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T3CON	; T3CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, ; No Synchronization, Prescale 1:1
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR3H	; TMR3H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR3L	; TMR3L = WREG = 0xF6, to do 10 pulses
	BCF	PIR2,TMR3IF	; Clear Timer3 Flag
	BSF	PIE2,TMR3IE	; Enable Timer3 Interrupt
	BSF	IPR2,TMR3IP	; Make Timer3 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T3CON,TMR3ON	; Start the Timer3
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

Program 9.25: Generate a 62.5% Duty Cycle square wave using Timer 3 with interrupts.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority ; 0x18 for Low Priority Interrupt

	GOTO	T3_ISR	; Go to Timer3 ISR
	ORG	0x100	; Set Origin 0x0100 for MTR3 module
T3_ISR	BCF	T3CON,TMR3ON	; Stop the Timer3
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	TSTFSNZ	0x00,0	; Check content of [0x00], if it is 0, ; it indicates end of -ve Level, ; If 0 got SL1
	BRA	SL1	; Branch to SL1
	MOVLW	0x60	; WREG = 0x60
	MOVWF	TMR3H	; TMR3H = WREG = 0x60
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR3L	; TMR3L = WREG = 0x00
	BSF	PORTB,4	; Set PORTB.4
	BRA	SL2	; BRA SL2
SL1	MOVLW	0xA0	; WREG = 0xA0
	MOVWF	TMR3H	; TMR3H = WREG = 0xA0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR3L	; TMR3L = WREG = 0x00
	BCF	PORTB,4	; Reset PORTB.4
SL2	BSF	T3CON,TMR3ON	; Start the Timer3
	RETfie		; Return from Interrupt Enable
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,4	; Set PortB Pin 4 as Output, TRISB.4=0
	MOVLW	0x01	; WREG = 0x01
	MOVWF	0x00,0	; File Reg [0x00] = WREG = 0x01
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T3CON	; T3CON = WREG = 0x84, 16-Bit Read/Write ; mode, T1OSC OFF, Internal Clock, ; Synchronization, without Prescale
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	BSF	PIE2,TMR3IE	; Enable Timer3 Interrupt
	BSF	IPR2,TMR3IP	; Make Timer3 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T3CON,TMR3ON	; Start the Timer3
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

9.2.5 Timer 4

The Timer 4 is always operated as an 8-bit timer and is identical to timer2. This is only present in PIC18F8720 and not in the other two devices PIC18F458 and PIC18C801. It

makes use of two timer registers TMR4 & PR4 and a control register – T4CON. The TMR4 register acts a preset register similar to TMR0L register of Timer0 and holds the current count value. The PR4 register is called as period register and is used to set the maximum ceiling count/time-period. The T4CON register provides different control/configuration bits that allows for the configuration and control of the Timer4. These three registers are readable and writeable.

The block diagram of Timer 4 is shown in fig 9.7 and the control bit details of the same is presented in the T4CON control register format.

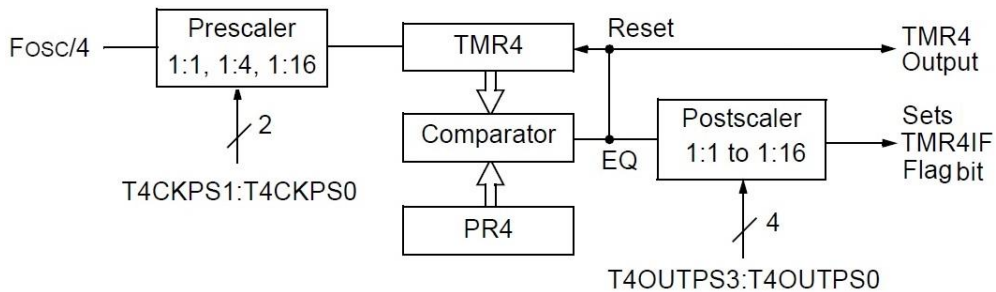


Fig 9.7 Block diagram of Timer 4

The Timer4 always operate based on main internal clock i.e. $F_{osc}/4$ and hence no control bits are provided to choose between internal or external clock. Similarly, no synchronization is required as the clock used is the internal clock and hence no control bit is provided related to synchronization.

Unlike Timer0, Timer1 and Timer3 the Timer2 and Timer4 include postscale select bits along with the prescale select bits. The timer pulses pass through both the prescale and postscale and hence no control bits are provided for enabling the prescale or postscale.

The prescale feature of timer4 is same as timer2 and is similar to Timer1/Timer3 and only provides division factors of 1, 4 and 16 to make the timer work similar to a Mod-4, or Mod-16 counter etc. that counts “Mod factor times the oscillator pulses”. The actual prescale value is set using the Timer4 clock prescale select bits – T4CKPS1:T4CKPS0.

The additional scaling feature present in timer4 is the postscale feature which is same as timer2 and similar to prescale. Here the timer4 output is applied to the postscale stage whereas in a prescale the input clock is applied to the prescale stage. This provides division factors of 1, 2, 3, 4, upto 16 to make the timer work similar to a Mod-N counter etc. that counts “Mod factor times the timer output pulses”. The actual postscale value is set using the Timer2 output postscale select bits – T4OUTPS3: T4OUTPS0.

T4CON: Timer4 Control Register Format

(Only PIC18F8720, not available on PIC18F458 & PIC18C801)

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	T4OUTPS3	T4OUTPS2	T4OUTPS1	T4OUTPS0	TMR4ON	T4CKPS1	T4CKPS0

bit 7

bit 0

Bit No	Bit	Description
bit 7	Unimplemented	Read as '0'
bit 6-3	T4OUTPS3:T4OUTPS0	Timer4 Output Postscale Select bits 0000 = 1:1 Postscale 0001 = 1:2 Postscale • • • 1111 = 1:16 Postscale
bit 2	TMR4ON	Timer4 On bit 1 = Timer4 is on 0 = Timer4 is off
bit 1-0	T4CKPS1:T4CKPS0	Timer4 Clock Prescale Select bits 00 = Prescaler is 1 01 = Prescaler is 4 1x = Prescaler is 16

Reading and Writing Timer Registers – TMR4 & PR4

The operation of timer4 is same as timer2 and similar to other timers. However, the setting of overflow flag is different from timer0, 1 and 3. Here the PR4 period register holds the ceiling/period value whereas TMR4 holds the preset/current count value. The number of pulses that are counted by timer4 is equal to the difference between the period register PR4 and the TMR4 preset/counter register. Hence the TMR4 and PR4 registers must be loaded with the respective values based on the required number of pulses.

The timer register TMR4 could be reset/cleared to 0 when the CCP/ECCP module is configured to use timer4 as its source for PWM mode.

Timing of Timer

The timer4 is started or stopped using TMR4ON bit of the T4CON register. Setting the TMR4ON bit to 1 starts the timer whereas clearing to 0 stops the timer.

Every PIC18 instruction takes a predefined number of cycles and a corresponding set of instructions of the program takes certain number of cycles. For an accurate timing the user must take this into account and adjust the value stored into timer4's TMR4 and PR4 registers. An equivalent example of the same is demonstrated in program 9.8 for timer0. The same approach could be applied to timer4 for determining the practical execution time.

Timer Overflow, Flag & Interrupt

The timer4 operates always in 8-Bit mode and flag is set when the timer count value in TMR4 matches the value in period register PR4. This is different to other timers 0, 1 and 3 which set timer flag when TMR value changes from 0xFFFF to 0x0000. When the PR4 and TMR4 match occurs the timer overflow flag TMR4IF is set.

An interrupt is generated when TMR4IF is set when the Timer Interrupt Enable – TMR4IE bit is set along with Global interrupt enable - GIE bit. Additionally, Peripheral interrupt enable – PEIE bit is required for all the timers except for Timer0.

The TMR4IF bit is accessed through PIR3 register, TMR4IE through the PIE3 register. The TMR4IE when set to 1 enables the timer4 interrupt and disables when it is set to 0. The GIE & PEIE bits are configured through the INTCON register. The GIE bit must be set to 1 to globally enable all the interrupts and PEIE must be set to 1 to enable all the peripheral interrupts. The Timer4 interrupt can be configured to high priority by setting the TMR4IP bit in the IPR3 register. The timer4 interrupt is set to low priority when the TMR4IP bit is cleared to 0.

Similar to other interrupts, the Timer4 flag i.e. TMR4IF bit must be cleared using software instructions as part of the Timer4's Interrupt Service Routine - ISR before the interrupt is re-enabled.

Registers related to Timer 4.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PR4	Timer4 Period Register							
TMR4	Timer4 Module Register							
T4CON	—	T4OUT PS3	T4OUT PS2	T4OUT PS1	T4OUT PS0	TMR4ON	T4CKPS1	T4CKPS0
INTCON	GIE/ GIEH	PEIE/ GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
IPR3	—	—	RC2IP	TX2IP	TMR4IP	CCP5IP	CCP4IP	CCP3IP
PIR3	—	—	RC2IF	TX2IF	TMR4IF	CCP5IF	CCP4IF	CCP3IF
PIE3	—	—	RC2IE	TX2IE	TMR4IE	CCP5IE	CCP4IE	CCP3IE

Steps to program Timer 4 without and with interrupts is similar to that of Timer2 except for the following.

- Timer4 is only present in PIC18F8720
- TMR4IF of PIR3 should be used for timer 3 in place of TMR2IF of PIR1 for timer 2
- TMR4IE of PIE3 should be used for timer 3 in place of TMR2IE of PIE1 for timer 2
- TMR4IP of IPR3 should be used for timer 3 in place of TMR2IF of IPR1 for timer 2

Program 9.26: Generate a square wave using Timer 4 for 12 pulses (50% duty cycle)

	#INCLUDE <P18F8720.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,5	; Set PortB Pin 5 as Output, TRISB.5=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T4CON	; T4CON = WREG = 0x00, No Prescale, ; Postscale
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR4	; TMR4 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR4	; PR4 = WREG = 0x0C, to do total 12 pulses
	BCF	PIR3,TMR4IF	; Clear Timer 4 Flag
	BTG	PORTB,5	; Toggle PORTB.5
	BSF	T4CON,TMR4ON	; Start the Timer4
AGAIN	BTFS	PIR3,TMR4IF	; Check if TMR4IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR4IF is not set.
	BCF	T4CON,TMR4ON	; Stop the Timer4
	BRA	HERE	; Branch to Here
	NOP		; No Operation
	END		

9.2.5.1 Programming Timer4 interrupts

The table 9.3 presents the flag and control bits for using the timer interrupts. The following program 9.27 and 9.28 presents example programs of timer4 with interrupts. A typical structure of Timer interrupts is shown in Fig 9.1.

Program 9.27: Generate a square wave using Timer 4 for 12 pulses (50% duty cycle) with interrupts.

	#INCLUDE <P18F8720.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T4_ISR	; Go to Timer4 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer4 ISR
T4_ISR	BCF	T4CON,TMR4ON	; Stop the Timer4
	BTG	PORTB,5	; Toggle PORTB.5
	BCF	PIR3,TMR4IF	; Clear Timer 4 Flag

	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR4	; TMR4 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR4	; PR4 = WREG = 0x0C, to do total 12 pulses
	BSF	T4CON,TMR4ON	; Start the Timer4
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,5	; Set PortB Pin 5 as Output, TRISB.5=0
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T4CON	; T4CON = WREG = 0x00, No Prescale, ; Postscale
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR4	; TMR4 = WREG = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	MOVWF	PR4	; PR4 = WREG = 0x0C, to do total 12 pulses
	BCF	PIR3,TMR4IF	; Clear Timer 4 Flag
	BSF	PIE3,TMR4IE	; Enable Timer4 Interrupt
	BSF	IPR3,TMR4IP	; Make Timer 4 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T4CON,TMR4ON	; Start the Timer4
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

Program 9.28: Generate a 62.5% Duty Cycle square wave using Timer 4 with interrupts.

	#INCLUDE <P18F8720.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T4_ISR	; Go to Timer4 ISR
	ORG	0x100	; Set Origin 0x0100 for MTR4 module
T4_ISR	BCF	T4CON,TMR4ON	; Stop the Timer4
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	TSTFSNZ	0x00,0	; Check content of [0x00], if it is 0, ; it indicates end of -ve Level, ; If 0 go to SL1
	BRA	SL1	; Branch to SL1
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR4	; TMR4 = WREG = 0x00
	MOVLW	0xA0	; WREG = 0xA0
	MOVWF	PR4	; PR4 = WREG = 0xA0

	BSF	PORTB,5	; Set PORTB.5
	BRA	SL2	; BRA SL2
SL1	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR4	; TMR4 = WREG = 0x00
	MOVLW	0x60	; WREG = 0x60
	MOVWF	PR4	; PR4 = WREG = 0x60
	BCF	PORTB,5	; Reset PORTB.5
SL2	BSF	T4CON,TMR4ON	; Start the Timer4
	RETFIE		; Return from Interrupt Enable
	ORG	0x200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,5	; Set PortB Pin 5 as Output, TRISB.5=0
	MOVLW	0x01	; WREG = 0x01
	MOVWF	0x00,0	; File Reg [0x00] = WREG = 0x01
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T4CON	; T4CON = WREG = 0x00, No Prescale, ; Postscale
	BCF	PIR3,TMR4IF	; Clear Timer 4 Flag
	BSF	PIE3,TMR4IE	; Enable Timer4 Interrupt
	BSF	IPR3,TMR4IP	; Make Timer 4 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T4CON,TMR4ON	; Start the Timer4
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	BRA	HERE	; Branch to Here
	END		

9.3 Using Timers as counters

As discussed, earlier timers may also be used as counters. When a timer is used as a counter, the pulses that are provided to the timer stages are provided always from an external clock input signal. Here in PIC18 devices they are provided through RA4/T0CKI or RC0/T1CKI pins. A timer always gets a periodic pulse whereas a counter could get periodic or non-periodic pulses. A typical application of position/speed measurement involves measurement of pulses created by a proximity sensor when the slotted rotor disc of a motor rotates. Another application may involve counting of objects detected by proximity sensor on a conveyor belt. An example that demonstrates the use of timer and counter together is presented in program 9.32.

Program 9.29: Count the number of pulses coming at RA4/T0CKI using Timer 0 and send the count to Port B and Port C continuously.

#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR

	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for Main Program
START	CLRF	TRISB,0	; Set PortB as Output, TRISB=0
	CLRF	TRISC,0	; Set PortC as Output, TRISC=0
	BSF	TRISA,4	; Set PortA pin 4 as input, TRISA.4=1
	MOVLW	0x28	; WREG = 0x28, 0x68 for Timer0 in 8-Bit mode
	MOVWF	T0CON	; T0CON = WREG = 0x28, Timer in 16-Bit ; mode, as counter with +ve edge input from ; RA4/T0CKI and No Prescale
	BCF	INTCON,TMR0IF	; Clear Timer 0 Flag
	CLRF	TMR0H	; TMR0H = 0x00
	CLRF	TMR0L	; TMR0L = 0x00
	BSF	T0CON,TMR0ON	; Start the Timer0
HERE	MOVFF	TMR0H,PORTB	; Move TMR0H value to PORTB
	MOVFF	TMR0L,PORTC	; Move TMR0L value to PORTC
	BRA	HERE	; Branch to Here
	END		

Program 9.30: Count the number of pulses coming at RC0/T1CKI using Timer1 and send the count to Port B and Port D when TMR2 reaches PR2 of 100d = 0x64h from 0x00.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for Main Program
START	CLRF	TRISB,0	; Set PortB as Output, TRISB=0
	CLRF	TRISD,0	; Set PortC as Output, TRISD=0
	BSF	TRISC,0	; Set PortC pin 0 as input, TRISC.0=1
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T1CON	; T1CON = WREG = 0x82, 16-Bit Read/Write ; Mode, T1OSC OFF, Ext Clock, ; Synchronization, without Prescale
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, without Prescale, ; postscale
HERE	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1H	; TMR1H = 0x00
	MOVWF	TMR1L	; TMR1L = 0x00
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	CLRF	TMR2,0	; TMR2 = 0x00

	MOVLW	0x64	; WREG = 0x64
	MOVWF	PR2	; PR2 = 0x64
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	CALL	MTR1	; Call MTR1 module
	MOVFF	TMR1H,PORTB	; Move TMR1H value to PORTB
	MOVFF	TMR1L,PORTD	; Move TMR1L value to PORTD
	BRA	HERE	; Branch to Here
	ORG	0x200	; Set Origin 0x0200 for MTR1 module
MTR1	BSF	T1CON,TMR1ON	; Start the Timer/Counter1
	BSF	T2CON,TMR2ON	; Start the Timer2
AGAIN	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR2IF is not set.
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BCF	T1CON,TMR1ON	; Stop the Timer1
	BCF	T1CON,TMR2ON	; Stop the Timer2
	RETURN		; Return from MTR1 Module
	END		

Program 9.31: Count the number of pulses coming at RC0/T1CKI using Timer1 and send the count to Port B and Port D when TMR2 reaches PR2 of 100d = 0x64h from 0x00 using interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x18	; Set Origin 0x18 for Low Priority interrupt
	GOTO	T2_ISR	; Go to T2_ISR
	ORG	0x200	; Set Origin 0x0200 for MTR1 module
T2_ISR	BTFSS	PIR1,TMR2IF	; Check if TMR2IF is set. Skip next inst if set.
	BRA	LAST	; Branch to Again, when TMR2IF is not set.
	BCF	T1CON,TMR1ON	; Stop the Timer/Counter1
	BCF	T2CON,TMR2ON	; Stop the Timer2
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	MOVFF	TMR1H,PORTB	; Move TMR1H value to PORTB
	MOVFF	TMR1L,PORTD	; Move TMR1L value to PORTD
	CLRF	TMR2,0	; TMR2 = 0x00
	MOVLW	0x64	; WREG = 0x64

	MOVWF	PR2	; PR2 = 0x64
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1H	; TMR1H = 0x00
	MOVWF	TMR1L	; TMR1L = 0x00
	BSF	T1CON,TMR1ON	; Start the Timer/Counter1
	BSF	T2CON,TMR2ON	; Start the Timer2
LAST	RETFIE		; Return from T2_ISR
	ORG	0x100	; Set Origin 0x0100 for Main Program
START	CLRF	TRISB,0	; Set PortB as Output, TRISB=0
	CLRF	TRISD,0	; Set PortC as Output, TRISD=0
	BSF	TRISC,0	; Set PortC pin 0 as input, TRISC.0=1
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T1CON	; T1CON = WREG = 0x82, 16-Bit Read/Write ; Mode, T1OSC OFF, Ext Clock, ; Synchronization, without Prescale
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T2CON	; T2CON = WREG = 0x00, without Prescale, ; postscale
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1H	; TMR1H = 0x00
	MOVWF	TMR1L	; TMR1L = 0x00
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	CLRF	TMR2,0	; TMR2 = 0x00
	MOVLW	0x64	; WREG = 0x64
	MOVWF	PR2	; PR2 = 0x64
	BCF	PIR1,TMR2IF	; Clear Timer 2 Flag
	BSF	PIE1,TMR2IE	; Enable Timer2 Interrupt
	BCF	IPR1,TMR0IP	; Make Timer 2 Low Priority
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	T1CON,TMR1ON	; Start the Timer1
	BSF	T2CON,TMR2ON	; Start the Timer2
HERE	NOP		; No operation, Alternate function
	BRA	HERE	; Branch to Here
	END		

Program 9.32: Count the number of pulses coming at RC0/T1CKI using Timer1 and store the count for every 1sec timed by timer 3 in memory and send the count to Port B and Port D using 25MHz crystal oscillator.

A 25MHZ crystal oscillator provides an instruction cycle time of 0.16 μ s. and maximum of 10.49msec in full range of timer 3. To achieve 1sec the full timer3 range should be repeated x times. The expression for x is given by

$$x = 1s / 0.16 \mu s = 6.25 * 10^6.$$

The number of pulse cycles provided by timer3 in one full range is 65536. Hence the number of times the full range of repetition of timer3 is

$$y = 6.25 * 10^6 / 65536 = 95.367$$

For better accuracy (and neglecting instruction cycle time) the full timer3 cycle range is not considered, only 62500 pulses are considered.

Hence if timer3 register TMR3H & TMR3L are loaded with 0x0BDC or 3036d (65535-62500+1) then

$$Y = 6.25 * 10^6 / 62500 = 100.$$

So, repeating the timer 3 for 100 times with pulse count of 65200 (TMR values of 0x0BDC) provides a time of 1sec.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x100	; Set Origin 0x0100 for Main Program
START	CLRF	TRISB,0	; Set PortB as Output, TRISB=0
	CLRF	TRISD,0	; Set PortC as Output, TRISD=0
	BSF	TRISC,0	; Set PortC pin 0 as input, TRISC.0=1
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T1CON	; T1CON = WREG = 0x82, 16-Bit Read/Write ; Mode, T1OSC OFF, Ext Clock, ; Synchronization, without Prescale
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T3CON	; T3CON = WREG = 0x82, without Prescale, ; postscale
	LFSR	0,0x10	; FSR0 = 0x10
HERE	MOVLW	0x64	; 100 Loop count to give 1sec.
	MOVWF	0x00	; File Reg [0x00] = 0x64 = 100d
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1H	; TMR1H = WREG = 0x00
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1L	; TMR1L = WREG = 0x00
	CALL	MTR1	; Call MTR1 module
	MOVFF	TMR1H,PORTB	; Move TMR1H value to PORTB
	MOVFF	TMR1L,PORTD	; Move TMR1L value to PORTD

	MOVF	TMR1L,0,0	; Get 16-Bit timer value, WREG = TMR1L, ; TMR1H = TMR1 High Byte
	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0+1
	MOVF	TMR1H,0,0	; WREG = TMR1H
	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0+1
	BRA	HERE	; Branch to Here
	ORG	0x200	; Set Origin 0x0200 for MTR1 module
MTR1	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BSF	T1CON,TMR1ON	; Start the Timer/Counter1
L1	BCF	T3CON,TMR3ON	; Stop the Timer3
	MOVLW	0x0B	; WREG = 0x0B
	MOVWF	TMR3H	; TMR3H = 0x0B
	MOVLW	0xDC	; WREG = 0xDC
	MOVWF	TMR3L	; TMR3L = 0xDC
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	BSF	T3CON,TMR3ON	; Start the Timer3
AGAIN	BTFSS	PIR2,TMR3IF	; Check if TMR3IF is set. Skip next inst if set.
	BRA	AGAIN	; Branch to Again, when TMR3IF is not set.
	DECFSZ	0x00,1,0	; Decrement File Reg [0x00], skip next inst if ; value is 0.
	BRA	L1	; Branch to L1 to do timer3 100 times.
	RETURN		; Return from MTR1 Module
	END		

9.4 Using Timers to determine number of instruction cycles

The number of cycles ($F_{osc}/4$) taken by instruction can be found or verified using a timer with interrupts. The instructions whose cycles need to be found may be included after the respective timer is started. When breakpoints are included for each of those instructions, the change in TMR1H, TMR1L indicates the number of cycles taken by that instruction.

Program 9.33: Determining number of cycles required for execution of an instruction.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority Interrupt ; and 0x18 for Low Priority Interrupt
	GOTO	T1_ISR	; Go to Timer1 ISR
	ORG	0x100	; Set Origin 0x0100 for Timer0 ISR
T1_ISR	BTG	PORTB,2	; Toggle PORTB.2
	BCF	T1CON,TMR1ON	; Stop the Timer1
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag

	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0xF6	; WREG = 0xF6
	MOVWF	TMR1L	; TMR1L = WREG = 0xF6, to do 10 pulses
	BSF	T1CON,TMR1ON	; Start the Timer1
	RETFIE		; Return from Interrupt Enable
	ORG	0x0200	; Set Origin 0x0200 for Main Program
START	BCF	TRISB,2	; Set PortB Pin 2 as Output, TRISB.2=0
	MOVLW	0x84	; WREG = 0x84
	MOVWF	T1CON	; T1CON = WREG = 0x84, 16-Bit Read/Write ; Mode, T1OSC OFF, Internal Clock, ; No Synchronization, Prescale 1:1
	MOVLW	0xFF	; WREG = 0xFF
	MOVWF	TMR1H	; TMR1H = WREG = 0xFF
	MOVLW	0x9C	; WREG = 0x9C
	MOVWF	TMR1L	; TMR1L = WREG = 0x9C, to do 100 pulses
	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BSF	PIE1,TMR1IE	; Enable Timer1 Interrupt
	BSF	IPR1,TMR1IP	; Make Timer 1 High Priority
	BSF	INTCON, PEIE	; Enable Peripheral Interrupts
	BSF	INTCON, GIE	; Enable Global Interrupts
	BSF	T1CON,TMR1ON	; Start the Timer1
HERE	NOP		; No operation, Logic of optional alternate ; function may be placed here.
;	..	..	..
T1	BTFSS	0x00,0,0	; Test 0 th bit in File Reg 0x00, skip next ; instruction if 0 th bit is set.
T2	TSTFSZ	0x01,0	; Test File Reg 0x01, skip next ; instruction if file reg 0x01 is 0.
	DECFSZ	0x02,0	; Decrement File Reg 0x02, [0x02] = [0x02]-1, ; skip next instruction if File Reg 0x02 is 0.
	CPFSEQ	0x03,0	; Compare File Reg 0x03, [0x02] <> WREG, ; skip next instruction if File Reg 0x03 and ; WREG are equal.
	BRA	T1	; Branch to T1
	BRA	T2	; Branch to T2
;	..	..	..
	BRA	HERE	; Branch to Here
	END		

Table 9.7: File registers, their values and respective instruction's cycles

File Reg	Value	BTFSS		TSTFSZ		DECFSZ		CPFSEQ		BRA	
		S/NS	Cycle	S/NS	Cycles	S/NS	Cycles	S/NS	Cycles	S/NS	Cycles
0x00	0	No Skip	1								
0x01	0			Skip	2						
0x02	2					No Skip	1				
0x03	0							No Skip	1	Skip	2
WREG	9C										
TMR Before			0x9D		0x9E				0xA0		0xA1
TMR After			0x9E		0xA0				0xA1		0xA3
0x00	1	Skip	2								
0x01	1			No Skip	1						
0x02	0					No Skip	1				
0x03	0							Skip	2	Skip	2
WREG	0										
TMR Before			0xA3				0xA5		0xA6		0xA8
TMR After			0xA5				0xA6		0xA8		0xAA
0x00	0	No Skip	1								
0x01	1			No Skip	1						
0x02	1					Skip	2				
0x03	0							Skip	2	Skip	2
WREG	0										
TMR Before			0xAA		0xAB		0xAC				0xAE
TMR After			0xAB		0xAC		0xAE				0xB0

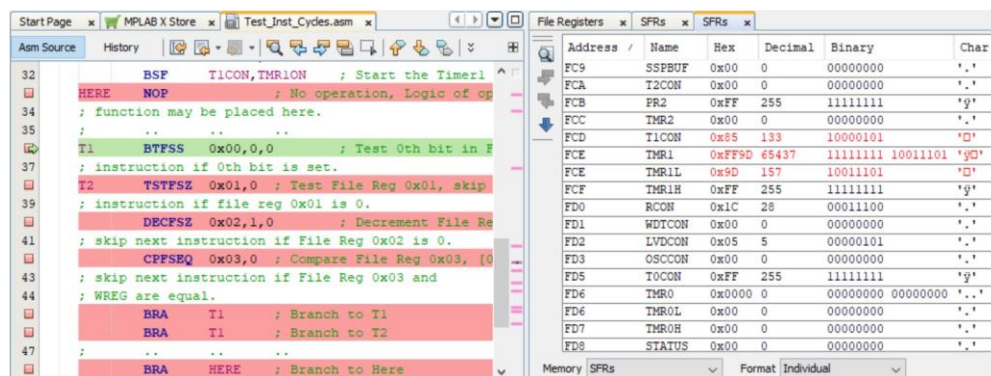


Fig 9.8 a) Before the execution of BTFSS for first set of values

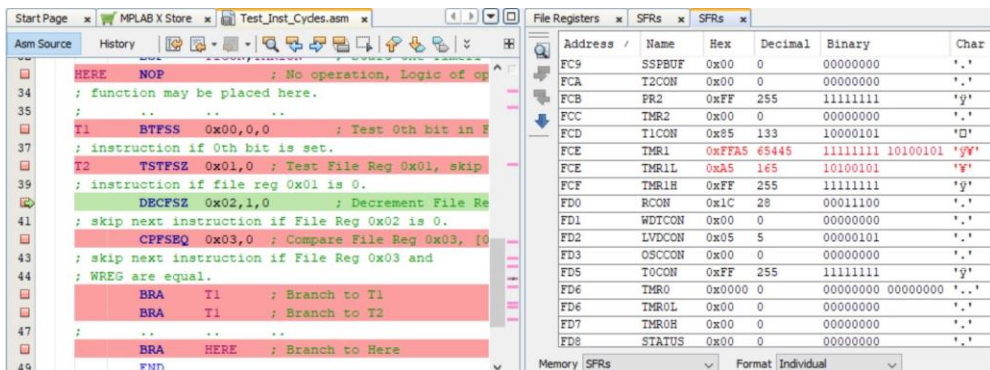
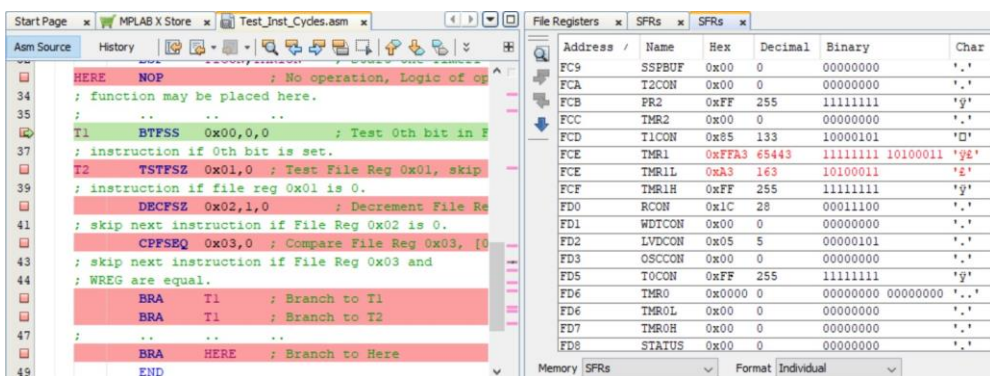
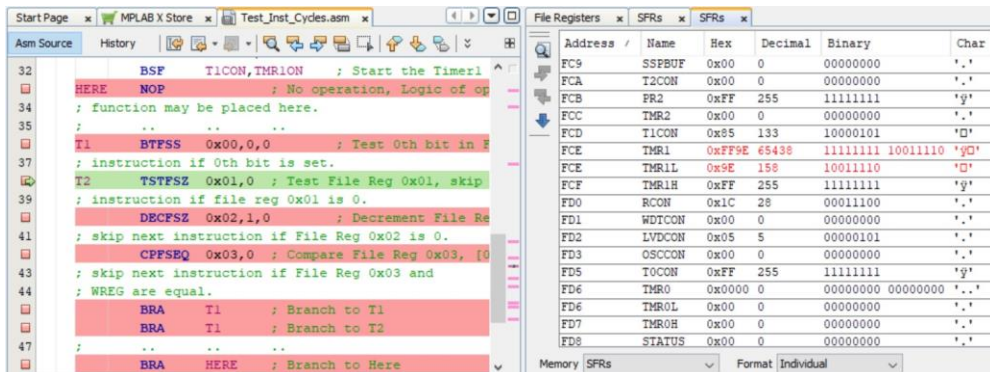


Fig 9.8 Simulation figure of MPLAB for Program 9.33

The program here is used to demonstrate the verification of cycles required for an instruction execution. This is the program 9.15 used to generate a square wave using Timer 1 for 10 pulses (100 pulses here) with interrupt. Additionally, instructions like BRA, BTFSZ, TSTFSZ, DECFNZ and CPFSEQ are included as alternate function in the infinite loop between

the two instructions HERE NOP and BRA HERE to find the number of cycles taken by the respective instructions for their execution for different conditions. The simulated information is shown in the figure 9.8. The same logic could be applied with any timer to determine the number of instruction cycles.